

TSFM-Lens: Controlled, Causal and Confirmatory Interpretability for Time-Series Foundation Models

Dan Kushnir

DAN.KUSHNIR@SPIALERT.COM

Abstract

Time-series foundation models (TSFMs) are now deployed zero-shot in forecasting pipelines, yet almost nothing is known about what they compute, and the tools that made language models inspectable do not transfer: TSFM tokens cover different spans of time from one architecture to the next, no ground truth exists for what a “concept” in a forecast should be, and the causal effects of internal features are small enough that uncontrolled measurements produce confident false positives. We present TSFM-LENS, an open-source system that turns a list of model checkpoints into a single, self-contained interpretability report with one command. It is built on three commitments. First, it is *controlled*: a leakage-audited synthetic benchmark whose generative components are known exactly, together with structured input perturbations, provides known answers against which every probe is validated. Second, it is *causal*: concepts are defined by what removing them does to the forecast, measured against row-matched random-direction nulls and gated by reach controls. Third, it is *confirmatory*: exploratory claims are registered by content hash and tested once on a sealed private split. Architecture-agnostic time alignment makes decoder-only, encoder-only and encoder-decoder models with token widths from 1 to 96 steps comparable, and every reported number carries a typed evidence class and a generated caveat. Applied to TimesFM, Chronos-2, Sundial and Chronos-Bolt, the system finds that the models’ causal features change forecast level and shape far more often than trend or seasonality; that the models agree on *which inputs* a concept groups (20 of 20 registered transfer claims confirmed on 800 held-out series) but rarely on *what it does* to those inputs (11 of 1,223 shared-input tests show the same causal effect); and that no concept is shared by all four models. We also document more than a dozen cases in which a plausible, well-formed result was wrong and the system’s own controls caught it. Code, configurations, a complete example run and a ledger of every finding with its reproduction pointer are public.

Keywords: mechanistic interpretability, time-series foundation models, sparse autoencoders, causal intervention, representation similarity, synthetic benchmarks, pre-registration, open-source software

1 Introduction

In the last three years, pretrained forecasting models have moved from research curiosities to default components. TimesFM (Das et al., 2024), the Chronos family (Ansari et al., 2024, 2025), Sundial (Liu et al., 2025), Timer (Liu et al., 2024), Moirai (Woo et al., 2024), MOMENT (Goswami et al., 2024), Lag-Llama (Rasul et al., 2023), Time-MoE (Shi et al., 2025) and Tiny Time Mixers (Ekambaram et al., 2024) are all released as open checkpoints and are used zero-shot on series they never saw. Their architectures differ a great deal: decoder-only stacks over 32-step patches, encoder-decoders over quantized single steps, encoders with cross-series attention, flow-matching heads that sample. Leaderboards say

which is more accurate on which benchmark. They do not say *what these models represent*, whether architectures this different end up representing the same things, or whether any internal structure we can name actually drives the forecast.

Mechanistic interpretability has produced a mature toolkit for exactly these questions in language models: probes (Alain and Bengio, 2017; Hewitt and Liang, 2019), representational similarity (Kornblith et al., 2019), activation patching (Vig et al., 2020; Meng et al., 2022; Zhang and Nanda, 2024), lenses (nostalgebraist, 2020; Belrose et al., 2023) and sparse dictionaries (Bricken et al., 2023; Cunningham et al., 2024; Gao et al., 2025), packaged in libraries such as TransformerLens (Nanda and Bloom, 2022), nnsight (Fiotto-Kaufman et al., 2025), pyvene (Wu et al., 2024) and SAEs (Bloom et al., 2024). Early work has begun to apply pieces of it to TSFMs (Wiliński et al., 2025; Mishra, 2026; Oublal et al., 2026; Divo et al., 2026). Three properties of forecasting models, however, break the direct transfer of these tools, and each of them causes silent errors rather than crashes:

- (i) **Tokens are not comparable across models.** One token is one time step in Chronos-T5, 32 steps in TimesFM, 16 in Chronos-2 and 96 in Timer. “Layer 6, position 10” refers to different stretches of time in different models, so every cross-model comparison needs an explicit, verified mapping from tokens to time.
- (ii) **There is no ground truth for concepts.** Text comes with linguistic annotations. Real series do not come with labels for their trend, seasonal period, changepoints or noise process, so a probe that “finds seasonality” cannot be checked against the answer.
- (iii) **Effects are small and easy to fake.** Removing a single sparse feature typically moves a forecast by a few hundredths of the context’s standard deviation, less than the reconstruction error of the dictionary itself (Section 7.4). At that scale, a precision mismatch, a patch the head never reads, or a null measured on the wrong rows produces a clean, confident and wrong result.

This paper presents TSFM-LENS, a system designed around these three problems, and uses it to study three research questions:

RQ1 (concepts). What interpretable, forecast-relevant concepts do TSFMs learn?

RQ2 (sharing). Are these concepts shared across models with different architectures, and in what sense: do the models respond to the same inputs, have the same effect, or both?

RQ3 (causality). Do the concepts causally affect the forecast, and how much of the forecast do they account for?

A supporting question, **RQ4**, asks where in depth these computations happen and how far such depth claims can be compared across architectures.

Approach. TSFM-LENS rests on three commitments that together distinguish it from existing tooling (Table 1): *controlled* measurement, in which a synthetic benchmark with exact generative ground truth and a battery of structured perturbations gives every probe a known answer to be validated against; *causal* concepts, in which a feature counts only if removing it changes the forecast by more than removing a random direction of the same size does, on the same series, with reach controls proving the intervention landed; and *confirmatory* testing, in which dev-corpus findings are frozen in a hash-pinned registry and

tested once on a sealed private split. Two further design choices make these commitments usable: architecture-agnostic *time alignment*, which measures each model’s token-to-time map and pools all models onto common windows, and a *one-command report* that explains every figure in plain language, states what each number can and cannot support, and renders every skipped analysis with its reason.

Contributions.

- C1. An open-source system** (TSFM-LENS, ~61k lines of analysis code, 171 test modules) that goes from a configuration listing model checkpoints to a self-contained HTML report through 19 fingerprinted stages, with a zero-code adapter path for new Hugging Face checkpoints (Section 4).
- C2. A controlled benchmark generator** with two explicit leakage tiers, a banded-DTW leakage audit, sealed dev/private splits with fresh epochs, and bit-exact regeneration (Section 4.2).
- C3. A causal concept pipeline** that defines concepts by their effect on nine forecast channels, pools them across models into an atlas, and separates two notions of sharing that prior work conflates: *same inputs* (top- k overlap against a stratum-matched null) and *same effect on the same inputs* (joint ablation against activation-matched floors, with a lower-tail test for disagreement) (Section 5.5).
- C4. A statistical protocol** in which the series is always the resampling unit, every comparison enters a multiplicity ledger whose attainable minimum p -value is checked, and confirmation happens once, on sealed data (Section 6).
- C5. An empirical study** of four TSFMs that answers RQ1–RQ4 with typed evidence and a held-out replication (Section 7).
- C6. A catalogue of measurement failures:** fourteen documented cases in which an uncontrolled analysis produced a plausible, well-formed and wrong result, each paired with the built-in control that caught it (Section 8). We argue this is the most transferable contribution: it shows concretely why interpretability pipelines for forecasting models need these controls.

Main findings. On TimesFM 2.5, Chronos-2, Sundial and Chronos-Bolt:

- **RQ1.** Causal sparse features are real (293 features across 29 layers clear a row-matched random-direction null) but overwhelmingly act on forecast *level* and *shape*. They tile a low-dimensional effect space continuously rather than forming discrete concepts: nine readable “families” cover 98% of them but do not beat a column-shuffle null.
- **RQ2.** The models agree on *which series* a concept groups: all 20 registered cross-model transfer claims replicate on 800 sealed series (top- k AUC 0.971–0.995). They rarely agree on *what the concept does*: only 11 of 1,223 shared-input tests show the same causal effect, the dominant sharing class is “convergent” (same effect, different inputs), and no concept has causal members in all four models. Seven pairwise similarity metrics rank the six model pairs inconsistently (Kendall’s $W = 0.28$), and representational similarity cannot detect lineage.

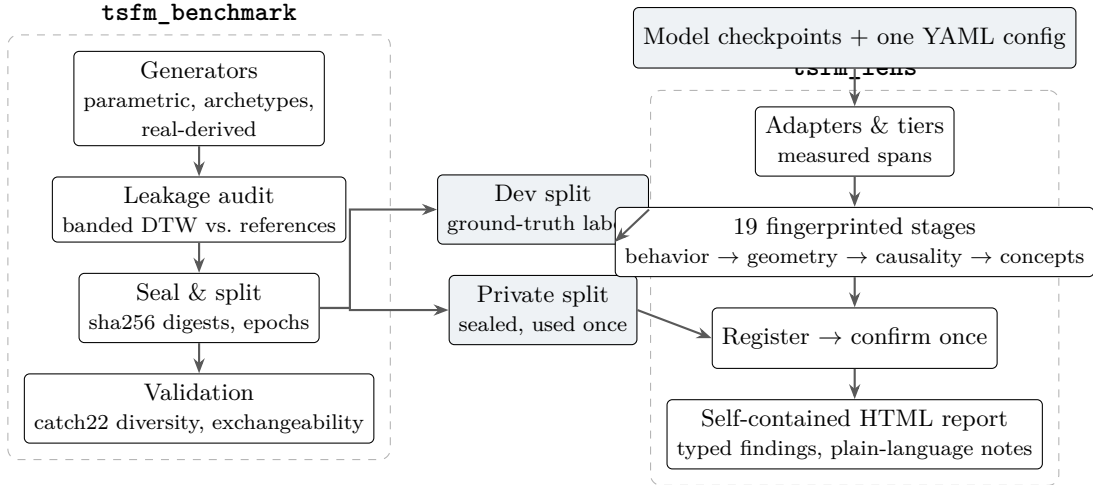


Figure 1: System overview. **tsfm_benchmark** produces leakage-audited, sealed corpora with exact generative ground truth; **tsfm_lens** takes a configuration naming model checkpoints and produces one report. Dev-split findings are exploratory; the private split is opened once, by the **confirm** stage, for claims registered in advance. The two packages install separately because their dependencies barely overlap.

- **RQ3.** Single features are causally load-bearing but small: the median feature effect is about 3.5 times smaller than the effect of replacing a layer by its SAE reconstruction, and 64% of visually prominent features have no measurable causal effect at all.
- **RQ4.** Where in depth models react to structured corruptions replicates on held-out data (9 of 9 registered corruptions for the TimesFM/Chronos-2 pair), but cross-architecture depth claims are qualified by how much of each model’s forward pass is actually observed (from 14% to 98% of forecast FLOPs).

How to read this paper. Sections 3–6 describe the system; Section 7 uses it; Section 8 shows what its controls prevent. We do not attempt to report every result the system has produced. The full ledger of findings, each with exact numbers, evidence class, confirmation status and a reproduction pointer, is `FINDINGS.md`; Appendix A indexes the entries cited here. The complete example run analyzed in Section 7, including its report and every per-stage artifact, is `examples/concept_atlas_v2`, and every data figure in this paper is regenerated from it by `paper/figures/make_figures.py`.

2 Background and Related Work

Time-series foundation models. TSFMs are transformers pretrained on large corpora of real and synthetic series and applied zero-shot. They differ in almost every design decision that matters for interpretability. TimesFM (Das et al., 2024) is a decoder-only stack over 32-step input patches that emits 128-step output patches. Chronos (Ansari et al., 2024) quantizes scaled values into a vocabulary and trains a T5 encoder–decoder (Raffel et al., 2020) with one token per step and a sampled decoder; Chronos-Bolt replaces the decoder

with a direct patch head, and Chronos-2 (Ansari et al., 2025) is an encoder over 16-step patches that adds cross-series “group” attention. Sundial (Liu et al., 2025) is decoder-only with a flow-matching head (Lipman et al., 2023) that samples forecasts. Timer (Liu et al., 2024) uses 96-step tokens, Lag-Llama (Rasul et al., 2023) reads a set of lagged values rather than a contiguous window, and Moirai (Woo et al., 2024), MOMENT (Goswami et al., 2024), Time-MoE (Shi et al., 2025) and TTM (Ekambaram et al., 2024) add further variation in patching, masking and mixture-of-experts routing. Most of these models normalize each series internally, in the manner of reversible instance normalization (Kim et al., 2022). This detail turns out to matter: an adapter that calls a model below its public API can silently skip that normalization (Section 8). Synthetic pretraining data are increasingly common (Kang et al., 2020; Dooley et al., 2023), and Wan et al. (2026) recently documented “forecast collapse” to flat predictions on weakly predictable targets, a behavior we also measure (PM-07, BM-06).

Interpretability of time-series models and TSFMs. Saliency methods for time-series classifiers were benchmarked by Ismail et al. (2020), who found them unreliable across architectures. Kalnāre et al. (2025) applied activation patching, attention saliency and SAEs to time-series *classification* transformers. For TSFMs, Wiliński et al. (2025) analyzed block-wise representational redundancy and steered forecasts with concept directions derived from synthetic data. Four recent studies train sparse autoencoders on a single TSFM: Mishra (2026) ablates TopK-SAE features of Chronos-T5-Large and reports a depth hierarchy of causal importance; Oublal et al. (2026) adapt SAEs to explain black-box time-series models; Divo et al. (2026) look for monosemantic features in pretrained TSFMs; and Yıldırım (2026) argue that PatchTST (Nie et al., 2023) does not rely on superposition. These studies each examine one model or one method. None of them compares architectures on a common time axis, validates probes against generative ground truth, separates input agreement from effect agreement across models, or tests findings on held-out data. TSFM-LENS is complementary: it is a system for doing all of these at once, and its case study reaches some conclusions that a single-model analysis cannot (for example, that multi-model concepts are mostly convergent rather than shared; Section 7.3).

Mechanistic interpretability methods and tooling. Linear probes (Alain and Bengio, 2017) need control tasks to separate what a representation encodes from what the probe learns (Hewitt and Liang, 2019; Belinkov, 2022). Activation patching and causal mediation (Vig et al., 2020; Meng et al., 2022; Geiger et al., 2021; Wang et al., 2023) are sensitive to metric and corruption choices (Zhang and Nanda, 2024; Heimersheim and Nanda, 2024) and can create illusory subspaces (Makelov et al., 2024). The logit lens and tuned lens (nostalgebraist, 2020; Belrose et al., 2023) read predictions out of intermediate layers. Sparse dictionaries (Makhzani and Frey, 2013; Bricken et al., 2023; Cunningham et al., 2024; Templeton et al., 2024; Gao et al., 2025) address superposition (Elhage et al., 2022), and crosscoders (Lindsey et al., 2024) learn features jointly across layers or models; SAE features are known to vary across training seeds (Paulo and Belrose, 2025). Libraries such as TransformerLens (Nanda and Bloom, 2022), nnsight (Fiotto-Kaufman et al., 2025), pyvene (Wu et al., 2024), SAELens (Bloom et al., 2024) and Captum (Kokhlikyan et al., 2020) make these methods easy to apply to one model. They are microscopes: they provide primitives, not comparative designs, nulls, multiplicity control or confirmation. TSFM-LENS uses the same primitives (hooks,

patching, TopK SAEs) but wraps them in the experimental design needed for comparative claims.

Representation similarity and universality. Whether different networks learn the same representations has been studied through convergent learning (Li et al., 2016), SVCCA and PWCCA (Raghu et al., 2017; Morcos et al., 2018), CKA (Kornblith et al., 2019), RSA (Kriegeskorte et al., 2008) and model stitching (Lenc and Vedaldi, 2015; Bansal et al., 2021), and has culminated in claims of universality (Olah et al., 2020) and convergence (Huh et al., 2024). Our results add a caution specific to TSFMs: similarity metrics rank model pairs inconsistently, a randomly initialized pair can look more similar than a genuinely related trained pair, and unit-level correspondence measures fail rotation controls because TSFM representations are extremely low-rank (Section 7.3).

Validity of empirical claims. Sanity checks for saliency (Adebayo et al., 2018), interpretability illusions (Bolukbasi et al., 2021; Makelov et al., 2024), the gap between attention weight and importance (Jain and Wallace, 2019) and ablation studies of individual units (Zhou et al., 2018) all show that interpretability outputs can look meaningful without being so. More broadly, adaptive reuse of data inflates error rates (Gelman and Loken, 2014; Dwork et al., 2015), leakage undermines ML-based science (Kapoor and Narayanan, 2023), results can fail to transfer to fresh test sets (Recht et al., 2019), and pre-registration is the standard remedy (Nosek et al., 2018). TSFM-LENS imports these remedies directly: registered hypotheses, a sealed split used once, specification curves (Simonsohn et al., 2020) for analysis-knob robustness, and resampling at the level of the independent unit (Efron and Tibshirani, 1993; Cameron et al., 2008).

3 Design Principles

TSFM-LENS is opinionated. Each principle below corresponds to a failure that actually happened during development; Section 8 lists them with their numbers. The principles are enforced in code (as gates, refusals and tests) rather than left as recommendations in documentation.

P1. Ground truth by construction. Every probe is first validated on a case whose answer is known. The benchmark records every generative component of every synthetic series (trend order and scale, seasonal periods, AR coefficients, changepoints, anomalies, intermittency), so “does the model represent seasonality?” can be checked against the truth before it is asked of real data. Every controlled perturbation (detrending, deseasonalizing, level shifts and six others) is applied to series whose components are known, so its effect on the input is known as well.

P2. Alignment is measured, not declared. A token’s time span is measured from the model’s own impulse response and checked against what the adapter declares. Cross-model analysis happens only on a common window axis (Section 4.4). A model whose tokens are not time-localized is routed to behavioral analysis only, with the verdict rendered in the report.

P3. A causal claim needs a null, a reach control and a same-seed baseline.

Every intervention is compared with a random intervention of matched size on the *same*

Capability	<i>TransformerLens</i>	<i>msight / pyvene</i>	<i>SAELens</i>	<i>Captum</i>	<i>TSFM SAE studies[†]</i>	<i>TSFM-Lens</i>
Hooks, caching, activation patching	✓	✓	○	○	○	✓
Sparse dictionaries and feature ablation	○	○	✓	✗	✓	✓
Time-series models without per-model engineering	✗	○	✗	○	✗	✓
Measured token→time alignment across architectures	✗	✗	✗	✗	✗	✓
Benchmark with exact generative ground truth, leakage audit	✗	✗	✗	✗	○	✓
Random-direction nulls and reach controls for every ablation	✗	✗	✗	✗	✗	✓
Cross-model concept matching (inputs vs. effects)	✗	✗	✗	✗	✗	✓
Series-level resampling and multiplicity ledger	✗	✗	✗	✗	✗	✓
Registered, one-shot confirmation on sealed data	✗	✗	✗	✗	✗	✓
Typed evidence class and generated caveat per claim	✗	✗	✗	✗	✗	✓
One-command, novice-readable report	✗	✗	○	✗	✗	✓

Table 1: Capabilities relevant to comparative TSFM interpretability. ✓ provided as a feature; ○ possible with user code or partially provided; ✗ not provided. General libraries are designed as single-model microscopes and are complementary to TSFM-LENS, which uses the same primitives. [†]Mishra (2026); Oublal et al. (2026); Divo et al. (2026); Yildirim (2026), judged from the papers.

series. Before any intervention is trusted, patching a layer into itself must change the forecast by exactly 0.0, and patching in a different layer’s activations must change it by a nonzero amount. Sampled models are seeded identically in every pair of compared calls.

P4. The series is the unit of inference. Windows within a series are not independent. All confidence intervals and p -values resample series (paired, cluster or validation-series bootstraps), all p -values are floored at $1/n_{\text{boot}}$, and every correction family’s attainable minimum is checked against its threshold.

P5. Explore, then confirm once. Everything computed on the dev split is exploratory. Hypotheses are frozen with the content hash of the artifact they came from, and the sealed private split is opened once. A second look is recorded as a repeated look, and a fresh epoch must be minted to confirm again.

P6. Every claim is typed, and failures are loud. Each finding carries an evidence class (BEHAVIORAL, DESCRIPTIVE, GEOMETRIC, LINEAR-TRANSLATABLE, CAUSAL WITHIN-MODEL, ILLUSTRATIVE, HELD-OUT CONFIRMED) and a caveat generated from that class. An unsupported capability is skipped with a reason rendered in the report. An un-resolvable slice disables the analysis rather than guessing, and a broken assumption raises.

P7. Accessible by default. The deliverable is one self-contained HTML file that a person with no interpretability background can read. Every figure has a caption and a “What

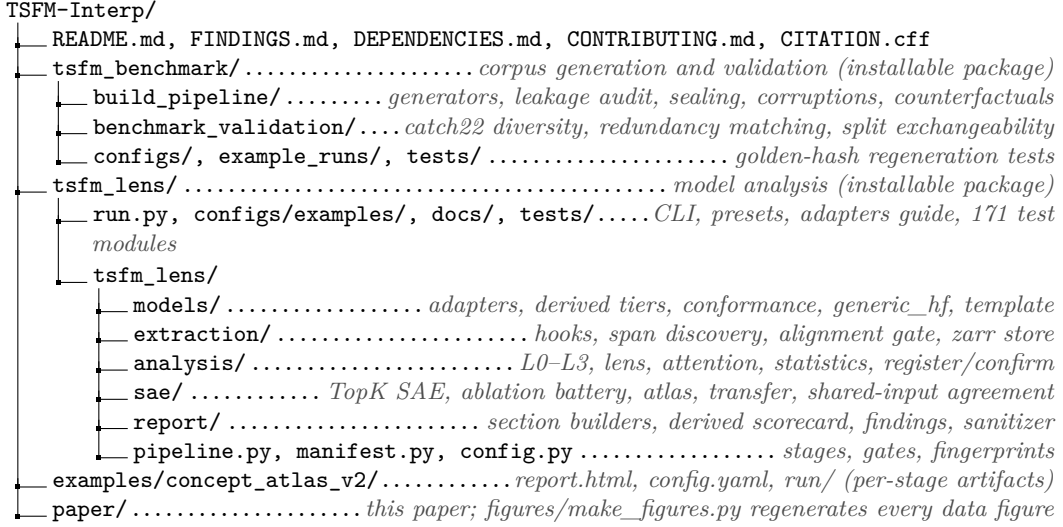


Figure 2: Repository layout of the `main` branch. The benchmark and analysis packages are independent; the example run folder contains every lightweight artifact of the run analyzed in Section 7, so its report and this paper’s figures can be regenerated without GPUs.

does this mean?” explainer that states the figure’s purpose, how to read it and its limitations. The headline scorecard is computed from artifacts by a pure function; no verdict in it is written by hand.

Two further engineering rules protect reproducibility. New behavior is opt-in, so that sealed corpora regenerate bit-exactly (golden hashes are pinned in the benchmark tests). And every stage’s configuration is fingerprinted, so a cached artifact computed under a different configuration is refused rather than silently reused.

4 System Architecture

4.1 Repository layout

The repository (Figure 2) contains two separately installable packages and a published example run. The `main` branch is the clean, publishable surface. A `dev` branch preserves the full research history: the research log from which `FINDINGS.md` is curated, standalone study drivers and experiment configurations that reproduce individual findings.

4.2 Controlled benchmark generation

Two leakage tiers. Real data enter the benchmark only as leakage references, as realism calibration and as raw material for explicitly labeled generators. The *synthetic* tier touches no real data and has exact ground truth: the `parametric` generator is additive (trend, multiple seasonalities, AR noise, changepoints, anomalies, random walk, intermittency and heteroskedasticity), and `random_parametric` draws a recipe from one of eight frozen archetypes (trend-dominant, seasonal-dominant, multi-seasonal, regime-switching, AR colored noise, anomaly-heavy, clean low-noise, noisy chaotic), with four more available opt-in.

The *real-derived* tier (`mixture`, `block_bootstrap` (Künsch, 1989) and `sequential_par`) inherits its source distribution, and its ground truth is attributional (which source, which blocks). Every sample is labeled with its tier. Mixing the tiers in one corpus is a deliberate design decision: “no leakage” and “realistic” pull in opposite directions, and the labels let each analysis report its results per tier rather than choosing one of the two.

Leakage audit and sealing. Every candidate sample passes a two-stage leakage gate against real reference collections (the Monash archive (Godaheva et al., 2021)): a cheap prefilter, then banded dynamic time warping (Sakoe and Chiba, 1978). It must also pass a cross-split near-duplicate check. Dev and private splits draw from disjoint seed ranges; seeds are mixed with SHA-256 (never a process-salted hash), and every sample is hashed so that `load_sealed(verify=True)` detects tampering. Fresh private *epochs* are minted by striding the seed space, so a consumed private split can be replaced without touching the dev split.

Validation. A separate package checks that a corpus is diverse and non-redundant. Redundancy is measured by shape matching (cross-correlation or DTW). Diversity is measured in the catch22 feature space (Lubba et al., 2019), never on a 2-D embedding, using the participation-ratio effective dimension (Gao et al., 2017), nearest-neighbor tails and near-collision rates overall and per group. Split exchangeability, on which the confirmatory design depends, is tested with an energy distance (Székely and Rizzo, 2013) plus per-feature TOST equivalence (Schuirmann, 1987), with a three-state verdict. On the v1 benchmark, dev and private are exchangeable (energy $p = 0.51$; 21 of 22 features equivalent; MP-09).

Controlled perturbations. Two perturbation families are built in. Nine structured *corruptions* (noise, detrend, deseasonalize, frequency shift, level shift, spike, smooth, warp, dropout) drive the L3 sensitivity and patching analyses; because the underlying components are known, their effect on the input is known exactly. *Generator counterfactuals* regenerate a series with one generative knob moved along a dose ladder while every random draw stays fixed; dose 1.0 must reproduce the original bit-exactly, a free identity test that caught a real bug (BM-05).

4.3 Model adapters and capability tiers

A model enters the system through an adapter (`tsfm_lens/tsfm_lens/models/base.py`) that exposes `load`, `predict` (point forecast plus requested quantiles, in the original scale), the underlying module, input preparation, a forward pass and `token_time_spans` (one contiguous time interval per token). Optional methods expose attention and MLP modules and attention patterns. The adapter’s *capability tier* is derived from which methods a subclass actually overrides, never declared: tier 0 (black box: load and predict), tier 1 (observable: hidden states with spans), tier 2 (steerable: single-pass context, so patching is possible) and tier 3 (decomposable: attention internals). A stage that needs a higher tier than a model has is skipped for that model, and the report names the missing method. A *missing capability* raises `CapabilityUnavailable`, a fact about the adapter; a *measured* failure of time localization raises `NotTimeLocalized`, a fact about the model, and routes it to behavioral analysis only.

Model	Checkpoint	Architecture	Token	Head	Tier	Params	FLOPs obs.
TimesFM 2.5	google/timesfm-2.5-200m	decoder-only	32	deterministic	3	231M	42.5%
Chronos-2	amazon/chronos-2	encoder (+group attn.)	16	deterministic	3	119M	97.5%
Sundial	thuml/sundial-base-128m	decoder + flow head	16	sampled	2	128M	22.4%
Chronos-Bolt	amazon/chronos-bolt-small	encoder + patch head	16	deterministic	2	48M	79.3%
Chronos-T5	amazon/chronos-t5-*	encoder-decoder	1	sampled	3	8M–709M	14.4% ^a
Timer	thuml/timer-base-84m	decoder-only	96	deterministic	2 ^b	84M	–
Lag-Llama	Lag-Llama	decoder over lags	lags	sampled	2	–	–

Table 2: Models with adapters. The first four form the case study of Section 7. “Token” is the number of time steps per input token; “FLOPs obs.” is the share of a full forecast’s FLOPs spent in captured blocks, measured with PyTorch’s FLOP counter (`budget/model_budget.json`). Sundial’s share is low because its sampled flow-matching head runs outside the captured pass. ^aChronos-T5-Base; only the encoder is captured, so the sampled decoder is never observed (DE-05). ^bVia the zero-code `generic_hf` adapter.

Adding a model takes one of three routes. The zero-code `generic_hf` adapter discovers layers and measures spans for a Hugging Face checkpoint and refuses rather than guesses. It has been verified on Timer, which it recovers with 96-step spans from measurement alone (MP-01). A hand-written adapter is a copy of `tsfm_lens/tsfm_lens/models/TEMPLATE_adapter.py` placed in `models/contrib/`, where it is discovered by an AST scan. In either case `run.py --check-adapter` runs a one-command checklist: patch-identity and patch-reach conformance, the alignment gate, span discovery and capability rows. The adapters guide (`tsfm_lens/docs/ADDING_A_MODEL.md`) documents the pitfalls we encountered, including a worked account of an adapter that silently bypassed the checkpoint’s input normalization (Appendix E). Table 2 lists the models used in this paper.

4.4 Time alignment across architectures

Cross-model analysis needs a common index. TSFM-LENS measures each token’s span from its impulse response (`extraction/span_discovery.py`): a perturbation injected at one time step should move the tokens whose spans contain it, and the resulting profile is gated on peak-to-pedestal contrast and contiguity. Given spans $I_j = [s_j, e_j)$ for tokens $j = 1, \dots, n$ and windows $W_w = [wL, (w+1)L)$ of width L (32 by default), the overlap-pooling matrix is

$$P_{wj} = \frac{|I_j \cap W_w|}{\sum_{j'} |I_{j'} \cap W_w|}, \quad \tilde{h}_w = \sum_j P_{wj} h_j, \quad (1)$$

and every cross-model analysis runs on the resulting `[series, window, dim]` tensors (Figure 3). A window that receives no token raises an error rather than producing a zero row. During extraction an *alignment gate* injects an impulse into each window and checks, layer by layer, that the matching pooled window moves most. Because coarse tokens cannot resolve fine windows (a 96-step token on 32-step windows can reach at most one third of diagonal hits), the gate normalizes hits by their attainable ceiling (MP-02). The impulse amplitude is self-calibrated per checkpoint and context length.

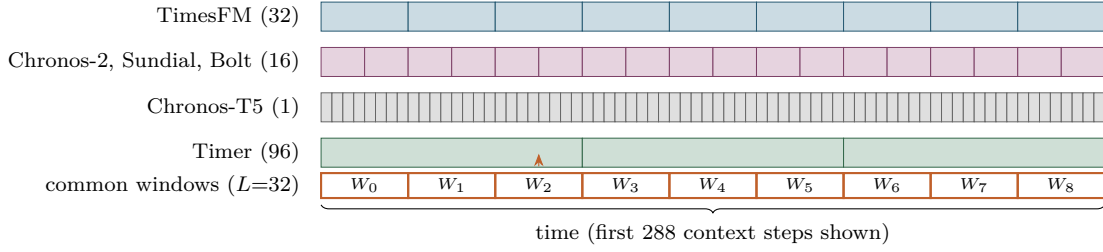


Figure 3: Architecture-agnostic time alignment. Tokens of each model cover measured, contiguous spans of different widths (Chronos-T5’s one-step tokens are drawn in groups of four); Equation (1) pools them by overlap onto common windows, so that all cross-model statistics compare the same stretch of time.

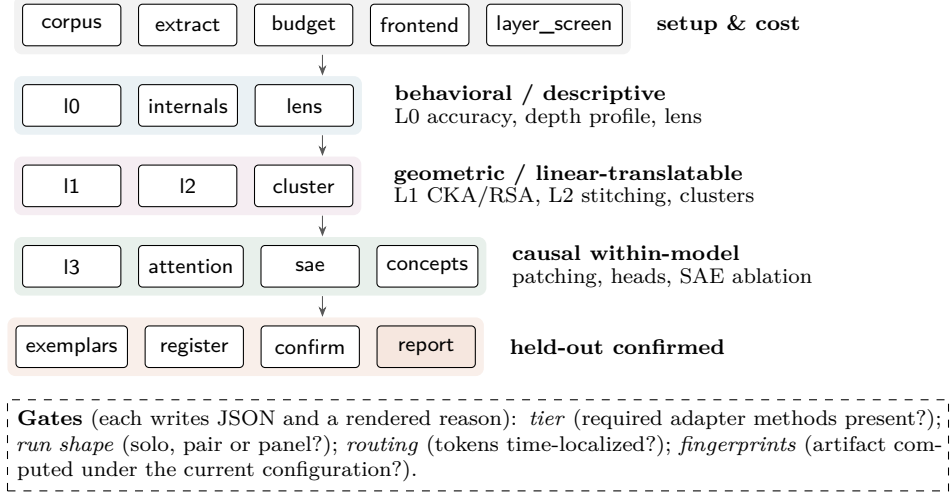


Figure 4: The 19-stage pipeline, grouped by the strongest evidence class each group can produce. Stages below a gate that cannot run for a model are skipped for that model with the reason rendered in the report. Only **confirm**, on the sealed private split, produces HELD-OUT CONFIRMED evidence.

4.5 The stage pipeline and its gates

An analysis is a directed pipeline of 19 stages (Figure 4), each writing typed artifacts into the run directory. Stages skip themselves when their artifacts exist, and each stage declares the configuration keys its artifacts depend on. These are fingerprinted, so a changed setting marks exactly the dependent artifacts stale and a stale artifact is refused. Three gates decide what runs and write their decisions to JSON, so the report can name them as skip reasons: the *tier gate* (does every model support this stage?), the *run-shape gate* (comparison-only stages are dropped for a single model) and the *routing gate* (models that are not time-localized receive behavioral analysis only). Extraction captures the residual stream into a chunked zarr store in float16 and runs the alignment gate. All interventions go through a single primitive (`hooks.token_patch`), which raises if the model silently splits a batch.

4.6 The report: interpretability for non-specialists

The deliverable of a run is a single HTML file with no external dependencies (`examples/concept_atlas_v2/report.html`; Appendix D shows excerpts). It is designed so that a reader with no background in interpretability can follow it, and so that an expert can audit every number.

- **Question-first structure.** After an “at a glance” block, seven parts pose plain questions (“Is the comparison fair?”, “How well does each model forecast?”, “Where does the forecast form?”, “Do the models represent things the same way?”, “What causally drives the forecast?”, “Features and concepts”, “Robustness and held-out confirmation”), and each stage section opens with a card stating its question, its method, what a good and a bad result look like, and what it cannot tell you.
- **Every figure explained.** Each figure has a caption plus a collapsible “What does this mean?” note giving its purpose, how to read it and its limitations; the example report contains 361 such notes across its 19 sections. A glossary, a methods appendix and a gallery of known failure modes are generated from the code.
- **Derived, auditable verdicts.** The scorecard (33 rows in the example) is produced by a pure reduction over artifacts with no model names hard-coded: each row shows the measured value, the reference it is compared with (a null, a floor, a ceiling or a baseline) and the rule that turns the two into a verdict, so a reader can apply a different rule to the same numbers. A missing reference renders as “not comparable”.
- **Typed findings.** Every claim is a **Finding** record with an identifier, stage, evidence class, technical text, a plain-language sentence and a caveat generated from the evidence class. The example run emits 126 findings (58 descriptive, 29 behavioral, 21 causal within-model, 12 linear-translatable, 4 geometric, 2 illustrative), serialized to `report/findings.json`.
- **Nothing silently missing.** A stage that did not run renders as “not run” with its gate reason; a stage that raised renders as a visible “section failed”. A multiplicity ledger lists every correction family (45 families and 3,599 comparisons in the example) and flags any family whose smallest attainable adjusted p -value cannot reach α .
- **Reading levels.** A toggle switches between a headline view (fairness card, accuracy and confirmed findings only), the standard view and a methods view with every detail expanded.

The complete analysis is one command:

```
python run.py --config configs/examples/large_4model.yaml
```

and smaller presets cover a CPU-only smoke test with mock models, a single real model and a pair (`tsfm_lens/configs/README.md`).

5 Measurement Methods

This section defines what each stage measures. Throughout, $x \in \mathbb{R}^T$ is a context of length T (512 in the case study), $y \in \mathbb{R}^H$ the true continuation ($H = 64$), $\hat{y}$ a model’s point forecast and $h^{(\ell)}$ the residual stream after block ℓ .

5.1 Behavior and cost (L0)

Accuracy is scored per series with MASE (Hyndman and Koehler, 2006),

$$\text{MASE}(\hat{y}, y) = \frac{\frac{1}{H} \sum_{t=1}^H |y_t - \hat{y}_t|}{\max(\frac{1}{T-m} \sum_{t=m+1}^T |x_t - x_{t-m}|, \epsilon_x)}, \quad (2)$$

with the seasonal lag m taken from ground truth where available, and a scale floor ϵ_x because on intermittent series the naive denominator collapses and MASE spuriously improves (MN-23). sMAPE, pinball loss (Gneiting and Raftery, 2007) and interval calibration are reported alongside. Model pairs are compared with a paired bootstrap over series, per family and overall, with Holm correction across all pairs and families (Holm, 1979). Each model’s repeat-run *noise floor* (two identical runs with different seeds) is measured so that a difference can be read in floor units; a deterministic model has a floor of zero, which the report states rather than using as a referee. The **budget** stage measures parameters by role, FLOPs with PyTorch’s `FlopCounterMode`, latency, memory and the share of forecast FLOPs spent inside captured blocks (Table 2). The **frontend** stage tests the tokenizer before any layer runs: scale equivariance (forecasting cx should give $c\hat{y}$), NaN handling and quantization resolution.

5.2 Geometry and linear translatability (L1, L2)

For aligned activations $A \in \mathbb{R}^{n \times d_a}$ and $B \in \mathbb{R}^{n \times d_b}$ over the same n series (mean-pooled over windows, then centered), linear CKA (Kornblith et al., 2019) is

$$\text{CKA}(A, B) = \frac{\|B^\top A\|_F^2}{\|A^\top A\|_F \|B^\top B\|_F}. \quad (3)$$

It is computed for every layer pair of every model pair, with a cluster bootstrap over series and a shuffled-series null that breaks correspondence while preserving each model’s marginal geometry. L2 fits ridge maps between layers and reports only the *gain* of the stitched prediction over an input-feature baseline, never a raw R^2 , because a large part of any layer is linearly predictable from the input alone. Activation clusterings are compared across models with adjusted mutual information (Vinh et al., 2010). These are GEOMETRIC and LINEAR-TRANSLATABLE claims; they say nothing about what the model uses.

5.3 Perturbation and patching (L3)

For each corruption c of the nine in Section 4.2, L3 records a sensitivity *fingerprint*, the relative activation change $\|h^{(\ell)}(c(x)) - h^{(\ell)}(x)\| / \|h^{(\ell)}(x)\|$ per layer, and a within-model *restoration curve*: the fraction of the forecast damage undone by patching the clean activations of layer ℓ (whole layer, or one window at a time) into the corrupted forward pass. Two models

are compared by rank-correlating their fingerprints on the relative depth range they share, never by transplanting activations between models. Restoration is CAUSAL WITHIN-MODEL; agreement between models is correlational.

5.4 Depth: lens, internals and coverage

A skip lens decodes the forecast from each layer through the model’s own head, and a tuned ridge lens (Belrose et al., 2023) fits a per-layer readout; the crystallization depth is the shallowest layer whose lens forecast is within tolerance of the final one. **internals** reports per-layer effective dimension (participation ratio), decodability of benchmark families and CKA to the input. **layer_screen** selects the layers that deserve expensive analysis with a cheap proxy (**work_bend**). Two design points qualify every depth claim. First, the depth axis is the fraction of the *whole* stack’s blocks, including uncaptured parts, so Chronos-T5’s last captured encoder block sits at 0.478 rather than 1.0 (DE-04). Second, the lens is withheld for a model whose head reads positions the patch never writes (**forecast_reads_patched_positions**), because such a lens returns a perfectly flat and meaningless curve (DE-03).

5.5 Causal concepts

Sparse dictionaries. For each target layer, TSFM-LENS trains a TopK sparse autoencoder (Makhzani and Frey, 2013; Gao et al., 2025) on token-level activations,

$$z(h) = \text{TopK}(W_e(h - b_d) + b_e), \quad \hat{h} = W_d z(h) + b_d, \quad (4)$$

with the dictionary size chosen by a search, an auxiliary loss that revives dead atoms and a minimum number of training steps (MP-03). Fidelity, dead rate and forecast preservation are reported per target. Because a single SAE seed is not reproducible (Paulo and Belrose, 2025), n_{seeds} replicate dictionaries are trained at the primary’s size to measure a within-model reproducibility ceiling (MP-04).

The ablation battery. A feature i is tested on the k series on which it fires most. For those series, the layer’s activations are replaced by the SAE reconstruction with z_i set to zero, and the forecast is compared with the forecast under the full reconstruction:

$$\Delta_i^{(c)}(x) = g_c(f(x; h^{(\ell)} \leftarrow \hat{h}_{-i})) - g_c(f(x; h^{(\ell)} \leftarrow \hat{h})), \quad (5)$$

where $f(x; h \leftarrow \cdot)$ is the forecast with layer ℓ patched and g_c is one of nine forecast *channels*: trend, seasonal strength, spectral centroid, level, dispersion, near- and far-horizon shape distance, MASE and flatness. Using the full reconstruction, not the clean forecast, as the baseline keeps the dictionary’s own reconstruction error from being charged to the feature. The *null* removes a random unit direction in code space, scaled to the mean magnitude the ablations remove, on the *same* series; the feature clears channel c if its mean $|\Delta_i^{(c)}|$ exceeds the 95th percentile of the null draws restricted to its own rows. The row matching matters: a feature whose top series happen to sit in a high-response regime would otherwise clear a pooled null on that alone. The channel vector used downstream is $v_i^{(c)} = \overline{\Delta_i^{(c)}}/q_{95}^{(c)}$, the signed mean effect in null units. The near- and far-horizon shape channels are distances (≥ 0) and are treated as undirected everywhere, including in generated text. Before any

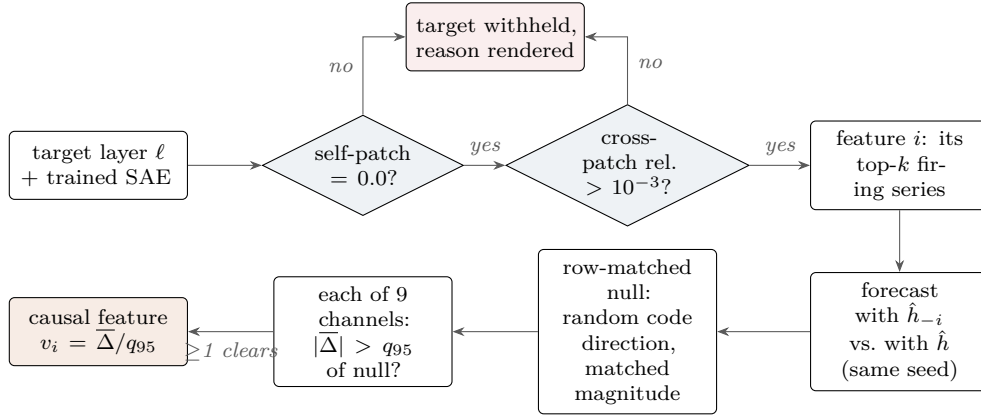


Figure 5: The ablation battery. A target is scored only after both reach controls pass; each feature is ablated on its own top-firing series and compared, channel by channel, with a random-direction null on exactly those series. Only features that clear at least one channel enter the concept analyses.

feature is scored, a *reach probe* requires that patching the layer’s own clean activations into itself changes the forecast by exactly 0.0 and that patching another layer’s activations changes it by a relative amount above 10^{-3} (Figure 5). A target that fails the probe is withheld with its reason, because a table of zeros from a patch that never lands reads as “these features do not matter”.

From features to concepts. Causal features are grouped at three granularities, each answering a different question. *Per-target concepts* cluster one layer’s feature vectors with k -means, choosing k by silhouette (Rousseeuw, 1987) among values whose smallest cluster has at least three members; a target with no admissible k is reported as *non-modular*. (Without the minimum-size constraint, silhouette rewards splitting off a single outlier; SH-20.) The *concept atlas* pools causal features from every model and layer and cuts a complete-linkage dendrogram at cosine 0.9, which guarantees that every pair of members has cosine at least 0.9; an atlas concept is a shared *causal-effect profile*, not a shared input. *Families* are a coarser, readable tiling: average linkage plus nearest-centroid assignment at cosine 0.5, with the number of families chosen by a parsimony rule and tested against a column-shuffle null. Families are titled deterministically from their directed channels (Figure 8).

5.6 Two notions of sharing

Whether a concept is “shared” between models is two separate questions, and conflating them produces wrong answers in both directions (Section 8). TSFM-LENS answers them separately and then combines them in an evidence ladder (Figure 6).

Same inputs. *Concept transfer* asks whether some feature in another model separates a concept’s top- k firing series from the rest, scored by AUC, against a null of random series subsets matched on benchmark strata (family and archetype); the test is run in both directions and a transfer is *reciprocal* if both legs clear. At atlas scale, p -values are corrected by Benjamini–Hochberg (Benjamini and Hochberg, 1995) per ordered model pair and leg.

L6	Confirmed on private data: registered claim confirmed once on the sealed split <i>held-out confirmed</i>
L5	Same causal effect on the same inputs: joint ablation on U beats matched floors <i>causal on shared inputs</i>
L4	Other dictionaries select the same inputs: reciprocal transfer vs. stratum-matched null <i>correlational</i>
L3	Reproducible: concept survives independent SAE seeds (within-model ceiling) <i>stability</i>
L2	Same inputs: the parts' top- k series overlap beyond chance (hypergeometric, BH) <i>correlational</i>
L1	Same forecast effect: atlas groups causal features of ≥ 2 models at pairwise cosine ≥ 0.9 <i>causal, compared</i>

Figure 6: The concept evidence ladder used by the report (computed by `concept_verdicts` in `report/derived.py`). Every causal feature has already passed the ablation battery (Figure 5). Each rung is a separate test with its own null, and the report states the highest rung each concept reaches together with the rule that produced its verdict. “Shared” in this paper means the **shared** sharing class: same effect (L1) and same inputs (L2).

When concept profiles compare what two concept parts fire on, input agreement is measured as the overlap of their top- k series (hypergeometric test with BH), never as a whole-series correlation, because two sparse features can have $\rho = 0.80$ with no shared top series (MN-19).

Same effect on the same inputs. *Shared-input causal agreement* takes every reciprocal-FDR transfer and ablates the feature (or concept part) on each side on the *same* series set U . Each side must first clear its own random-direction null on U to be scorable. Two statistics, level concordance and the cosine of the shape effects, must then beat both sides’ *activation-matched random-feature floors*: sets of equally active features whose ablation defines what agreement looks like by chance. A pair is “same causal effect” if both statistics beat both floors, “level only” or “shape only” if one does, and “acts differently” only if a statistic falls below the floors’ 5th percentile; failing to beat the 95th percentile is “no specific agreement”. The distinction matters: absence of agreement is not evidence of disagreement (MN-18).

Sharing classes. Each atlas concept then receives a `sharing_class`: *shared* (same effect and same inputs across models), *partially shared*, *convergent* (same effect, different inputs) or *single-model*. A concept part is flagged provenance-driven when at least 80% of its top series come from one real-derived generator.

6 Statistical Protocol

A system that runs thousands of comparisons per report must control how often it is fooled. TSFM-LENS does so with four mechanisms, summarized in Table 3.

The series is the resampling unit. Windows of one series share its trend, noise process and scale, so treating them as independent overstates precision. Every interval and p -value in TSFM-LENS resamples whole series (Efron and Tibshirani, 1993; Cameron et al., 2008):

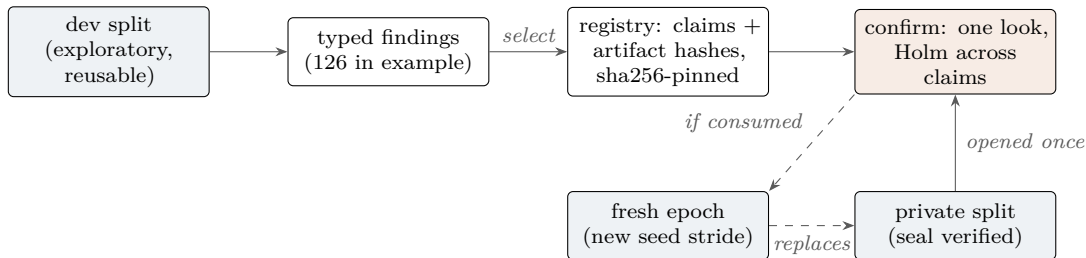


Figure 7: The confirmatory protocol. Only claims registered before the private split is opened can be confirmed; a second look is recorded, and replicating again requires a newly minted epoch.

a paired bootstrap for accuracy differences, a cluster bootstrap for CKA, a validation-series interval (probes held fixed) for stitching and a paired cluster bootstrap for patching. Train/validation splits are also made by series and sampled by stratum, never by slicing the head of a corpus that is stored grouped by task.

A multiplicity ledger. Every test is assigned to a named correction family (Holm for confirmatory families, Benjamini–Hochberg for discovery-oriented atlas transfers). Resampled p -values are floored at $1/n_{\text{boot}}$ (Phipson and Smyth, 2010), which means a Holm family of m tests cannot produce an adjusted p below m/n_{boot} . The ledger computes this attainable minimum for every family and flags families in which no test could ever be significant, a failure mode that grows silently as models are added (the number of pairwise comparisons is quadratic). The example run has 45 correction families and 3,599 comparisons; its most stringent L0 family (30 tests over 6 model pairs) has a minimum attainable Holm p of 0.015 at $n_{\text{boot}} = 2000$.

Noise floors and power. Effects are read against reference values that come from the run itself: repeat-run noise floors for accuracy, shuffled-series nulls for CKA, input-feature baselines for stitching, random-direction nulls and activation-matched floors for ablations, and replicate-seed ceilings for SAE reproducibility. Before the private split is opened, the confirm stage computes each registered claim’s minimum detectable effect at the private sample size by simulation (for example, a ΔMASE of 0.155 at $n = 240$ with 80% power), so that a non-replication can be read as “absent” or “underpowered”.

Register, then confirm once. The **register** stage freezes hypotheses drawn from dev results into a registry, together with the content hash of each dev artifact a hypothesis came from; the registry itself is hashed. The **confirm** stage refuses to run if a hash does not match, requires the private split’s seal to verify, tests each claim once and refuses to overwrite its own artifact. Forcing a rerun is recorded as **repeated_look**, and a consumed split is replaced by minting a new epoch (Figure 7). This is pre-registration (Nosek et al., 2018) applied to a software pipeline, and a guard against adaptive overfitting of the holdout (Dwork et al., 2015). Findings that were never registered cannot be rescued after the fact.

Claim type	Test statistic	Null / reference	Correction	Held-out
Model A more accurate (per family)	paired Δ MASE, series bootstrap	zero difference; repeat-run noise floor	Holm over all pairs $\times$ families	✓
Layers share geometry	linear CKA, cluster bootstrap	shuffled-series null at the same layer pair	per model pair	✓
Layer linearly translatable	ridge stitching gain	input-feature baseline (gain = 0)	per pair	–
Depth reaction agrees	Spearman ρ of fingerprints on shared depth	bootstrap CI of ρ	per corruption	✓
Layer restores forecast	restoration fraction	self-patch = 0, cross-patch $\neq$ 0	per model	–
Feature is causal	mean $ \Delta^{(c)} $ on own rows	row-matched random-direction q_{95}	per channel (count vs. chance)	–
Concept reproducible	best-match cosine across seeds	within-model replicate ceiling	–	–
Same inputs	top- k AUC in other model	stratum-matched random subsets	BH per ordered pair and leg; Holm when registered	✓
Same effect on same inputs	level concordance; shape cosine	activation-matched feature-set floors (q_{95} agree, q_{05} differ)	per test	–
Families are real	number and coverage of families	column-shuffle null	–	–

Table 3: Every claim type the case study reports, the statistic that tests it, the reference it must beat, its multiplicity correction and whether the **confirm** stage currently re-tests it on private data. Claim types without a held-out mark are reported as exploratory.

7 Case Study: Four Time-Series Foundation Models

7.1 Setup

We analyze TimesFM 2.5, Chronos-2, Sundial and Chronos-Bolt (Table 2) with the preset `tsfm_lens/configs/concept_atlas_v2.yaml`. The dev corpus has 965 series in five generator families (`parametric`, `random_parametric`, `mixture`, `block_bootstrap`, `sequential_par`); the real-derived families draw on Monash weather, ETT (Zhou et al., 2021) and the Chronos electricity and traffic collections. Contexts are 512 steps, the horizon is 64 and the alignment window is 32. SAEs are trained on 29 target layers (TimesFM: 10, at stride 2; Chronos-2: 7; Sundial: 9; Chronos-Bolt: 3), with three replicate seeds each for stability, and $n_{\text{boot}} = 2000$ throughout. The private split is 800 sealed series (epoch 0 of the v2 benchmark), opened once. The whole run took 6.43 hours on a single NVIDIA RTX A5000 (24 GB), of which SAE training took 2.39 hours and the concept stage 3.72 hours (Appendix C). All artifacts are in `examples/concept_atlas_v2/run`; results that come from other runs recorded in the ledger are marked with their `FINDINGS.md` identifier.

Before turning to the research questions, Table 4 summarizes the models’ behavior, which later sections use as context. Chronos-2 is the most accurate on the dev corpus

Model	median MASE	80% coverage	scale resid. ($\times 0.001$)	NaN input	latency
Chronos-2	1.1251	0.772	0.0093	handled	21 ms
TimesFM	1.1829	0.795	2.0×10^{-6}	propagates	38 ms
Sundial	1.3335	0.369	0.0332	propagates	7 ms
Chronos-Bolt	1.4913	0.749	0.0087	handled	14 ms
naive / seasonal naive	2.6829 / 2.0280				

Table 4: Behavioral character on the v2 dev corpus (965 series). Scale residual: error of $f(cx)/c$ against $f(x)$ in context standard deviations. Held-out, Chronos-2 is more accurate than TimesFM overall (paired Δ MASE -0.148 $[-0.211, -0.088]$, $p = 0.0005$, $n = 800$). Source: 10/summary.json, 10/calibration.json, frontend/frontend.json, confirm/confirmation.json.

and on held-out data. Sundial covers only 37% of outcomes with its nominal 80% interval. TimesFM and Sundial propagate an injected NaN to the forecast without raising, while the Chronos models handle it. Behind the Sundial row is a bug found during this work: the first Sundial adapter bypassed the checkpoint’s input normalization, and the `frontend` stage’s scale-equivariance residual (38.43 context standard deviations, against ≤ 0.009 for the other models) was at first read as a property of the model (PM-15; Section 8). After the fix the residual is 0.033.

7.2 RQ1: What forecast-relevant concepts do TSFMs learn?

Causal features exist, and the battery discriminates. Across the 29 targets, 293 SAE features clear the row-matched random-direction null on at least one of the nine forecast channels. Twenty-eight targets passed both reach controls (for example, TimesFM `stacked_xf.10`: self-patch exactly 0.0, cross-patch relative reach 0.146). The twenty-ninth, Chronos-2’s final block, was withheld: patching another layer into it moved the forecast by exactly 0, because Chronos-2’s head reads forecast placeholder tokens that a patch of the context positions never reaches (DE-03). Without the reach probe this target would have reported a full table of zero effects (`sae/concept_stage.json`). The battery is selective rather than permissive: on the reference four-model run, (feature, channel) cells clear the null at 2.49 to 6.61 times the rate expected by chance, and features nominated at random clear at about half the rate of features nominated by activation rules (MP-07).

The concepts are about level and shape. Figure 8 shows the nine effect families that tile the causal features. The three largest (148 of 288 assigned features) act on forecast level and volatility: *Level lowerers* (80), *Level raisers & Accuracy improvers* (61) and *Volatility dampeners* (48). Families whose defining channel is trend or seasonality are smaller: *Trend boosters & Accuracy improvers* (23), *Trend dampeners* (18), *Seasonality amplifiers* (17) and *Trend boosters & Volatility amplifiers* (12). Nearly every family also moves the near- and far-horizon shape distances. Level is not the whole story, however: on the reference run the median share of a causal feature’s effect that is a pure level shift is 0.535, and most causal features still clear a null after the level shift is removed from both the feature and the null (CA-07).

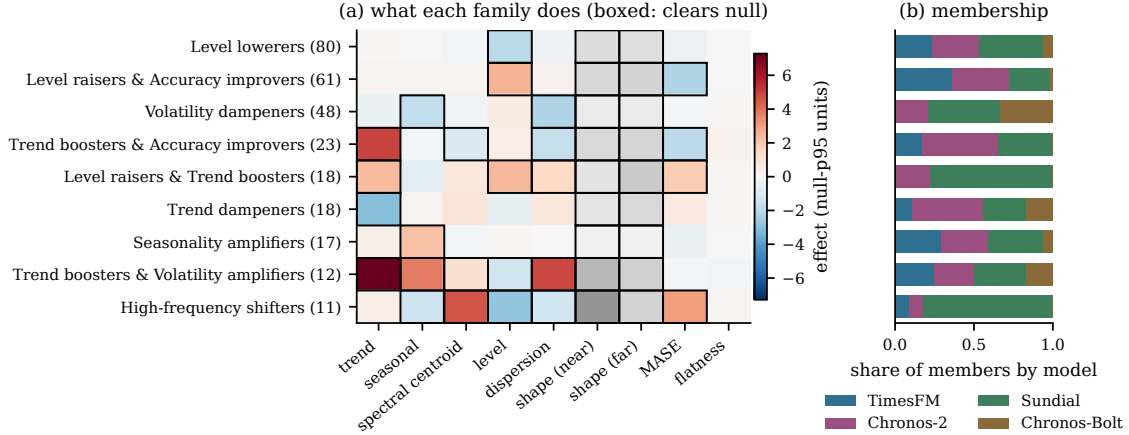


Figure 8: The nine causal-effect families of the example run, ordered by size. (a) Family centroid in null units: what removing a member does, negated to state what the feature *does*; red raises the channel and blue lowers it. Boxed cells clear the null in the family description. The near- and far-horizon shape channels are distances, drawn unsigned in gray. (b) Membership by model: families are populated unevenly by model. Source: `sae/concept_families.json`; regenerated by `paper/figures/make_figures.py`.

The concepts are graded, not discrete. Three observations indicate that “concept” should be read as a region of a continuous effect space rather than as a discrete unit. First, clustering within a layer rarely finds modules: 24 of 29 targets are non-modular, yielding only 9 per-target concepts. Second, the strict atlas assigns only 86 of 293 features (29%) to its 27 concepts. Third, the nine readable families cover 288 of 293 features (98%) but do *not* beat a column-shuffle null (permutation $p = 1.0$ for the number of families and $p = 0.985$ for coverage). A similar picture holds on the reference run, where silhouette is 0.31–0.35 for every family count from 3 to 16 (CA-10), and the 9-channel ablation fingerprints have a participation-ratio dimension of only 2.09 (MN-07). The families are therefore a useful vocabulary for reading the report, and the paper uses them in that sense only.

Concepts named after ground truth are rare, and naming them requires a null. Because the benchmark records every generative component, TSFM-LENS can ask whether a feature tracks a known field. Some do: after dead-feature revival, 154 of 10,240 TimesFM features and 115 of 6,144 Chronos-T5-Base features match a ground-truth field beyond a permutation null, the best being a Chronos feature tracking intermittency at $\rho = 0.881$ (MP-06). But raw correlations do not separate trained from untrained weights (an untrained twin scores 0.565 against 0.567), so only the margin over the null is evidence. Two tempting stronger claims failed their tests. Validating concepts by moving a generator knob and watching the concept respond was a confirmed negative, twice: 0 of 98 and then 0 of 151 knob–concept pairs survived correction, because a knob moves any active feature about as much as the targeted one (MN-17). And a vivid TimesFM “random walk” concept (19 of 20 top series from the random-walk archetype; ablation worsens MASE by 4.04 times the null) failed the seed-stability test and is not promoted (PM-11).

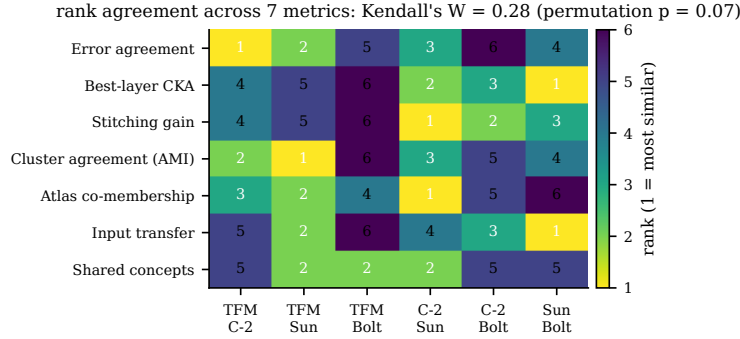


Figure 9: Rank of each model pair (columns) under seven similarity metrics (rows); 1 is most similar. Behavioral, geometric, translational, clustering and concept-level metrics disagree about which models are alike. Source: `report/model_similarity.json`.

Answer to RQ1. TSFMs contain sparse, causally effective directions, and what they control is mainly the level, volatility and shape of the forecast, with trend and seasonality handled by smaller groups. These directions form a continuum with a few dominant axes rather than a set of discrete, named concepts, and they are not specific to the generative factors a practitioner would name.

7.3 RQ2: Are concepts shared across architectures?

Geometry is shared above chance, but “similarity” has no single meaning. Peak linear CKA between TimesFM and each other model is 0.38–0.43, against a shuffled-series null of 0.037, and the TimesFM/Chronos-2 peak replicates on private data (dev 0.4262; private 0.4253 [0.4162, 0.4423]). The same above-null band holds across five unrelated architectures, including Timer through the zero-code adapter (0.449, null 0.063) and Lag-Llama (SH-01). The three models with 16-step tokens are far more similar to each other (0.909–0.928). Figure 9 shows, however, that seven pair-similarity metrics rank the six pairs inconsistently (Kendall’s $W = 0.28$, permutation $p = 0.07$). TimesFM/Chronos-2 is the most similar pair by error agreement (Spearman 0.947 across series), and only fourth by CKA and stitching gain. TSFM-LENS therefore never pools the metrics into one similarity score. Two further controls limit what similarity means. An untrained pair of the same architecture is more CKA-similar (0.878) than a genuinely related trained pair (0.734; difference $p = 0.0005$), so representational similarity cannot detect model lineage (MN-01). And an attractive unit-level correspondence measure rose, rather than fell, when both bases were randomly rotated (0.148 to 0.331), because TSFM layers are so low-rank (participation ratio as low as 1.54) that every direction correlates with the same few components (SH-19).

Models agree on which inputs a concept groups. Concept transfer asks whether another model has a feature that picks out a concept’s top series. At atlas scale, 1,223 of 1,581 transfer tests (77%) are reciprocal after Benjamini–Hochberg correction. Twenty claims were registered from dev results alone and tested once on the 800 private series: all 20 were confirmed under Holm correction (maximum adjusted $p = 0.0100$; private AUC 0.971–0.995 against null 95th percentiles of 0.57–0.77; Figure 11b). This result has now

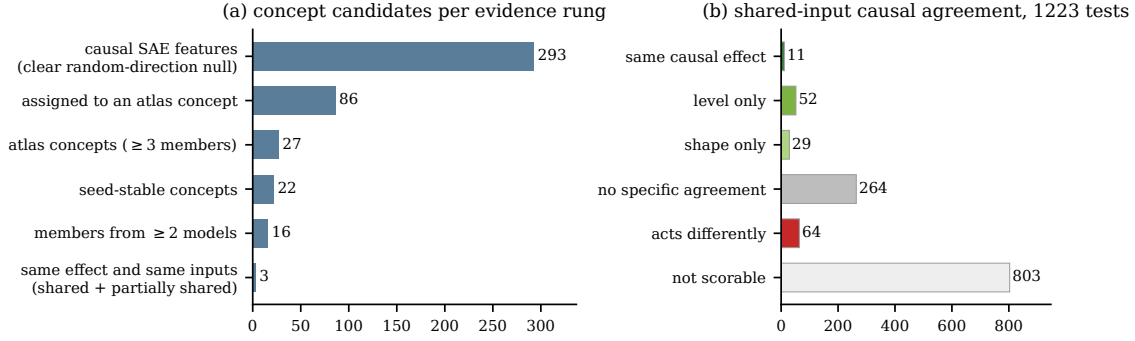


Figure 10: (a) Number of concept candidates at each rung. The first three bars count features and concepts in sequence; the last three are separate properties of the 27 atlas concepts. (b) Verdicts of the shared-input causal agreement test over all 1,223 reciprocal-FDR transfers; “acts differently” requires a lower-tail result against matched floors. Source: `sae/concept_stage.json`.

replicated on three independent private draws: 19 of 20 in search mode on v1 epoch 1, 20 of 20 with frozen feature pairs on epoch 2, and 20 of 20 on v2 (SH-18). Two caveats bound it. It is an *input-level* claim (the same series are grouped by both models), not a causal one. And the 20 v2 claims come from only two concepts, with only 6 of 20 private best features identical to the dev ones, so the replicated statement is that the destination layer has *some* feature that selects the same series.

Models rarely agree on what a concept does. Figure 10 follows the candidates up the evidence ladder. Of the 27 atlas concepts, 16 contain causal members from at least two models, but only two are *shared* (same effect and same inputs) and one is partially shared. The dominant class is *convergent* (13 concepts): features in different models have the same causal effect on *different* series. Eleven concepts belong to a single model. Restricted to the 22 seed-stable concepts, the counts are 9 convergent, 10 single-model, 2 shared and 1 partially shared. No concept has causal members in all four models; one spans three. The direct test agrees. Of the 1,223 reciprocal transfers, 420 are scorable (both sides clear their own null on the shared series), and among those only 11 (2.6%) show the same causal effect, 81 agree on level or shape alone, 264 show no specific agreement, and 64 (15.2%) act measurably differently. The family memberships in Figure 8b tell the same story: the families are segregated by model.

Answer to RQ2. Different TSFMs *detect* the same things: a concept learned by one model selects series that another model’s dictionary also selects, reliably enough to replicate on sealed data. They mostly do not *use* them the same way: effects shared on the same inputs are rare, the typical multi-model concept is convergent, and no concept is common to all four models. Whole-layer similarity metrics cannot reveal this distinction, and they disagree with each other.

7.4 RQ3: Do concepts causally affect the forecast, and how much?

The ablation battery establishes causality at the level of single features: every concept in Section 7.2 is built from features that move the forecast more than a matched random removal does, on the series they fire on, with reach verified. How much of the forecast these features account for is a separate question, and the answer is: individually, little. On the reference run, the median effect of removing one feature is 0.0395 context standard deviations, while replacing the layer by its SAE reconstruction already moves the forecast by 0.139, 3.5 times more (MN-15). Of 565 features that the report would otherwise display as “interesting” because they activate strongly, 364 (64.4%) clear no causal channel at all (MN-14); TSFM-LENS collapses them rather than rendering them as findings. Causality also depends on the dictionary: reviving dead atoms, which improves fidelity, reduced ground-truth alignment at every TimesFM depth tested (from -0.030 to -0.132) while the causal signal fell much less, and rose in one Chronos-2 layer (2.27 to 4.06 times chance; MN-21).

Head-level analyses show the same gap between what looks important and what is. The correlation between the attention mass a head places on seasonal lags and the seasonal power lost when that head is ablated is $\rho = 0.026$ in TimesFM and $\rho = -0.118$ in Chronos-T5-Base (CA-04), in line with the language-model finding that attention weights are not explanations (Jain and Wallace, 2019). A minimal circuit restoring seasonality under `deseasonalize` consists of one head in TimesFM ($p = 0.04$) but five non-additive heads in Chronos-T5-Base ($p = 0.008$; CA-03).

Answer to RQ3. Yes, causally, in the controlled sense: individual features and heads change the forecast in specific, null-beating ways. But no single feature carries much of the forecast, most prominent features carry nothing, and attention patterns are unreliable guides to which components matter. Causal claims about TSFM concepts are claims about small, distributed contributions.

7.5 RQ4: Where in depth, and how comparable is depth?

The corruption fingerprints locate where each model’s activations react to a structural change of the input, and their cross-model agreement is one of the most robust results in the ledger. For the registered pair TimesFM/Chronos-2, all nine per-corruption agreements replicate on private data (overall $\rho = 0.510$ [0.489, 0.527]; Figure 11a): the two models react at *matching* depths to detrending, deseasonalizing, frequency shifts and dropout, and at *opposite* depths to noise ($\rho = -0.742$), spikes (-0.780), warps (-0.658) and smoothing (-0.466). Whole-layer restoration can hide depth structure: in TimesFM it is flat across depth, yet per window it concentrates in the window just before the horizon and intensifies with depth (for **noise**, from 0.068 at layer 0 to 0.412 at layer 16; DE-01).

Comparing depth *locations* across architectures, however, needs two qualifications that TSFM-LENS computes and renders for every run. The captured share of a forecast’s computation ranges from 22% (Sundial, whose sampled head runs outside the captured pass) to 98% (Chronos-2), and for Chronos-T5-Base, where 100% of encoder blocks are captured, it is only 14% (DE-05). And on a whole-stack depth axis, Chronos-T5’s last encoder block sits at 0.478, so the comparable depth range with TimesFM is only half the axis (DE-04).

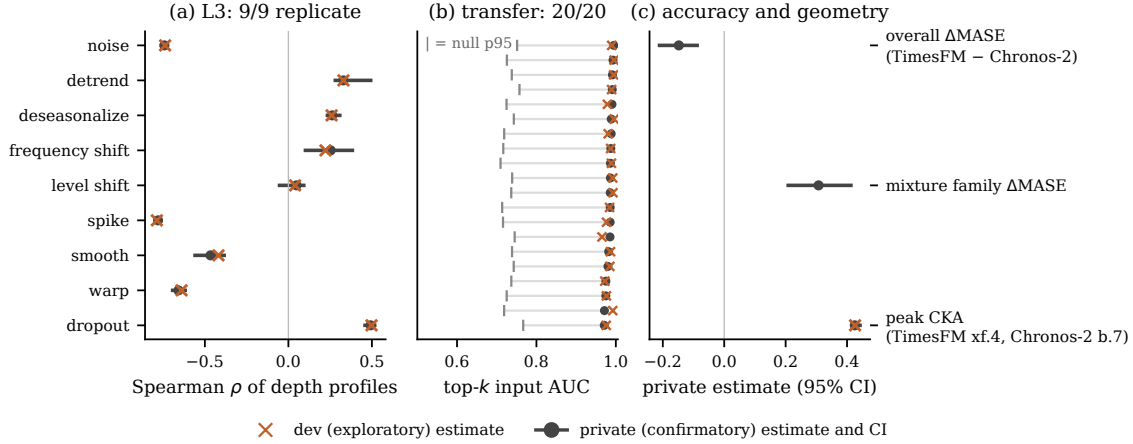


Figure 11: Every claim re-tested on the 800 sealed private series. (a) Per-corruption depth-profile agreement between TimesFM and Chronos-2; (b) the 20 registered concept-transfer claims, with each claim’s private null 95th percentile; (c) accuracy and peak CKA. Crosses: dev estimates; dots and bars: private estimates with 95% series-bootstrap intervals. Source: `confirm/confirmation.json`.

Which layers are “interesting” also depends on the selector: a specification curve over three selectors found no selected layer robust for any model (DE-08). Across a five-size Chronos-T5 scaling ladder (8M to 709M parameters), only family decodability increased monotonically with size (DE-07).

Answer to RQ4. Depth-resolved reaction profiles are reproducible and reveal which structural properties two models process at the same relative depth. Absolute depth locations are comparable across architectures only after accounting for captured compute and for the depth convention, which TSFM-LENS reports alongside every depth claim.

7.6 What replicated on held-out data

The `register` stage recorded 47 hypotheses, 31 of which the `confirm` stage can currently re-test (L2 stitching, archetype-level accuracy and clustering claims are registered but not yet replicable, and the report lists them as such). Every re-tested claim held (Figure 11): the registered accuracy claim (Chronos-2 more accurate on `mixture`, Δ MASE 0.307 [0.209, 0.413], $p_{\text{Holm}} = 0.0005$, $n = 240$, where the minimum detectable effect had been 0.155), the peak-CKA pair, all nine perturbation agreements and all 20 concept-transfer claims. Table 5 collects the headline results with their evidence class and status. What did *not* undergo held-out testing matters equally: the causal concept results of Sections 7.2–7.4, including the sharing classes and the shared-input agreement counts, are exploratory. They were, however, reproduced on a second corpus: on the v1 reference run, 0 of 19 atlas concepts span all four models, convergent is again the most common class, and 12 of 89 scorable shared-input tests act differently (SH-16, CA-06).

RQ	Result (example run unless an ID is given)	Evidence class	Status	ID
1	293 causal features; 9 families, largest act on level/volatility	causal within-model	exploratory, 2 corpora	CA-10
1	Families cover 98% but do not beat a shuffle null ($p = 1.0$)	descriptive	exploratory, 2 corpora	CA-10
1	Knob-specific concept mediation: 0/98, then 0/151	correlational	confirmed negative	MN-17
2	Peak CKA 0.425 vs null 0.037	geometric	held-out confirmed	SH-01
2	7 similarity metrics disagree ($W = 0.28$)	mixed	exploratory	SH-05/06
2	Untrained pair more similar than related trained pair	geometric	confirmed negative	MN-01
2	Concept transfer on inputs: 20/20	correlational	held-out confirmed (3 draws)	SH-18
2	No 4-model concept; convergent is the modal class	causal, compared	exploratory, 2 corpora	SH-16
2	Same effect on same inputs: 11/420 scorable	causal on shared inputs	exploratory, 2 corpora	CA-06
3	Feature effect $3.5\times$ below SAE reconstruction cost	causal within-model	exploratory	MN-15
3	64% of prominent features causally null	causal within-model	exploratory	MN-14
3	Attention mass uncorrelated with head causality	causal vs. descriptive	exploratory	CA-04
4	Depth-profile agreement, 9/9 corruptions	causal within-model	held-out confirmed	CA-01
4	Captured compute 14%–98% qualifies depth claims	descriptive	confirmed	DE-05
L0	Chronos-2 more accurate than TimesFM (-0.148)	behavioral	held-out confirmed	PM-05

Table 5: Headline results with evidence class and confirmation status. Each ID links to its entry in `FINDINGS.md`, which gives exact numbers, the reproduction command and the artifact path.

8 Controls Change Conclusions

How can a reader know that an interpretability system is better than a careful script that runs the same methods? For a tool whose purpose is to find out what is true about models, one useful answer is a record of cases in which the obvious analysis produced an answer, the answer was wrong, and a specific control caught it. Table 6 lists fourteen such cases from the development of TSFM-LENS. Each row gives what the uncontrolled analysis reported, what the controlled analysis found and the control that now runs by default. In every case the uncontrolled output was well-formed, plausible and, in several cases, more interesting than the truth. None of them raised an error.

Three failure classes. The cases fall into three classes, and each class maps onto a design principle of Section 3. *Interventions that never landed* (rows 1–4) produce flat curves, tables of zeros or perfectly matching numbers, which read as strong negative results. The remedy (P3) costs almost nothing: patching a layer into itself must return exactly 0.0, patching

#	Analysis	Uncontrolled result	Controlled result	Built-in control	ID
<i>Interventions that never landed or were not identical</i>					
1	Skip lens, Chronos-2	MASE byte-identical (1.564247) at all 12 layers: “forecast formed at layer 0”	The head reads placeholder tokens the patch never writes; real reach by depth 0.737, 0.643, 0.282, 0.095, 0.000	reach probe; lens withheld when the head does not read patched positions	DE-03
2	Patching with bf16 cache	Aggregates within 0.66% of CI width of the corrected values	Self-patch was 0.00214, not 0; 71% of role assignments at one layer changed	self-patch must be exactly 0.0; clean cache with autocast off	MN-24
3	Sampled model (Sundial) as causal destination	Scorable in 1 of 71 tests: “Sundial is causally inert”	Null reseeded differently from its baseline; after the fix, scorable in 12 of 71	identical seeds across every compared pair of calls	PM-09
4	Scale equivariance, Sundial	Residual 38.43 context s.d. read as a model property; 100% flat forecasts on electricity	Adapter bypassed the checkpoint’s normalization; residual 0.030, flat share 0.08	frontend residual flagged against peers	PM-15
<i>Nulls that were too weak, or asked the wrong question</i>					
5	Concept transfer	Uniform random-subset null: 30 of 30 concepts transfer	Stratum-matched null: 56.3% forward, 46.1% reciprocal	null matched on benchmark strata	SH-14
6	Cross-model causal comparison	18 of 19 activation-matched roles “fire together, act differently”	Same series, own nulls, matched floors: 12 of 89 scorable tests act differently	shared-input agreement with matched floors	CA-06
7	Disagreement test	109 of 288 tests “act differently” (failing a q_{95} bar)	Lower-tail test: 9 act differently; 32 show no specific agreement	q_{95} required for disagreement	MN-18
8	Random-weight twins as causal floor	One floor per model	TimesFM’s twin is an exact identity stack yet reaches the forecast (0.187); Chronos twins are numerically dead ($< 10^{-7}$); floor exists for 1 of 6 pairs	within-run matched-feature floors; relative reach gate	PM-08
9	SAE names from ground truth	11 of 11 headline features named after corpus provenance	After residualizing, 82 of 88 cards (93%) ranked float-rounding noise (ρ 0.203 $\rightarrow$ 0.615)	residual-scale floor; control bounded from both sides	MN-05
<i>Metrics that did not measure what they were named for</i>					
10	CKA and stitching for lineage	Related trained pair shows high similarity (CKA 0.734)	Untrained same-architecture pair is higher (0.878; $p = 0.0005$)	random-init twin comparison	MN-01
11	Unit-level correspondence	0.24–0.54, every CI excludes 0	Rotating both bases <i>raises</i> it (0.148 $\rightarrow$ 0.331)	rotation control on real activations	SH-19
12	Captured-depth chart	Chronos-T5 fully observed (1.0), TimesFM 0.851	Share of forecast FLOPs: Chronos-T5 14.4%, TimesFM 42.5%	FLOP-based coverage from the full forecast	DE-05
13	Span refusal statistic	“Diffuse” spans: refuses 3 of 5 correct adapters	Statistic grew with token count; peak-to-pedestal contrast accepts all (IoU 1.0)	gates normalized by attainable ceiling	MP-01
14	Single-seed SAE as gold standard	Layer selector beats random on 2 of 3 architectures	Verdict flips with the extraction seed; 3-seed average agrees across seeds	replicate SAE seeds	MP-04

Table 6: Fourteen documented cases in which an uncontrolled analysis gave a well-formed wrong answer and a built-in control changed the conclusion. Every control listed runs in the default configuration. Full accounts, including the exact commands, are in `FINDINGS.md` under each ID.

another layer must return a nonzero change, two unpatched predictions must be identical, and the input front end must behave like the peers’. *Nulls that were too weak* (rows 5–9) produce inflated positive results, and the failure often looks like a success: 30 of 30 concepts transferring, 93% of headline features “explained”. The remedy is a null matched on the right confound and measured on the same rows, bounded from both sides, with the lower tail used for claims of difference. *Metrics that did not measure what they were named for* (rows 10–14) are caught only by validating the metric on a case whose answer is known (P1): an untrained twin, a random rotation, a checkpoint whose spans are known, a replicate seed.

Smaller failures of the same kind. The ledger records further cases of each class. A headline result that an SAE reconstruction *improves* on the clean forecast ($-0.346 \Delta\text{MASE}$) replicated in 1 of 5 seeds and was retracted (MN-16). A layer-spacing rule rejected the top-scoring TimesFM layer and admitted two below the mean (DE-08). A sign convention applied to a distance channel generated the sentence “decreases how much the forecast changes” for every family, which is meaningless for a quantity that cannot be negative (CA-10). And a small grounded language model asked to narrate features produced a new class of unsupported claim after each of four guard-strengthening passes; in the final pass 2 of the 4 summaries that passed every guard were wrong, while 6 of 6 deterministic sentences were correct (MN-06). TSFM-LENS therefore composes every report sentence deterministically from measured values.

What this does and does not show. Table 6 is not a benchmark in which TSFM-LENS beats a baseline on a metric. It is a set of counterexamples: for each row, a pipeline that applies the same method without the control would have reported the uncontrolled result. We regard this as the strongest evidence available that the controls are necessary. It is necessarily incomplete: the table can list only the failures that were caught, and Section 10 discusses what the controls cannot catch. Every control in the table runs in the default configuration, and the project’s verification standard (CONTRIBUTING.md) requires each new test’s load-bearing assertion to be shown failing against a deliberately planted regression before it is merged.

9 Using and Extending TSFM-Lens

Workflow. A typical user installs the two packages, builds or downloads a benchmark, and runs one configuration:

```
pip install -e . && pip install -e tsfm_lens
cd tsfm_benchmark && PYTHONPATH=. python3 example_runs/run_full.py \
    --config configs/example.yaml --out ./benchmark_out --references monash
cd ../tsfm_lens && python run.py --config configs/examples/medium_2model.yaml
```

The CLI also provides `--doctor` (preflight checks), `--list-configs`, `--stages` and `--force` for partial reruns, `--check-alignment` and `--check-adapter` for new checkpoints or library upgrades, and `--verify-provenance`, which compares a run’s recorded environment with the current one. A CPU-only smoke configuration with mock models runs every stage in minutes and is part of continuous integration. The worked example (`tsfm_lens/docs/worked_example.md`) walks through a report section by section.

Cost. The full four-model analysis of Section 7 takes 6.4 hours on one 24 GB GPU. Most of the time goes to SAE training and the causal concept battery; stages up to and including L3 finish in about 16 minutes. Single-model and two-model presets are proportionally cheaper, and every stage can be disabled in the configuration.

Adding a model. The zero-code path handles checkpoints that load through `transformers.AutoConfig`; of eight TSFM checkpoints probed, two (Timer and Time-MoE) resolved this way, and five that ship standalone packages (Toto, MOMENT, Lag-Llama, Moirai, TTM) need a hand-written adapter (MN-25). The adapter template, the one-command checklist and the adapters guide are designed for that case. The guide’s checklist covers input normalization, seeded sampling, precision, batch chunking, layer regexes, span contiguity, encoder–decoder capture, reach controls and random-weight twins (Appendix E).

Extending the analysis. New sections are added as report builders with a declared artifact requirement; a missing artifact renders as “not run” and an exception as “section failed”, so an extension cannot silently drop content. New findings use the `Finding` record, which forces an evidence class. New configuration fields declare which stage artifacts they affect, so fingerprints stay correct.

10 Limitations and Threats to Validity

Scope of the models. The case study covers four models, all transformers with contiguous patch tokens. Models whose tokens read disjoint lags (Lag-Llama) defeat contiguity-based span discovery and are analyzed with reduced scope (MN-25). The analysis is univariate by design: Chronos-2’s cross-series group attention has no time interval and is not measured, so every Chronos-2 claim concerns its time axis only (PM-13). Chronos-T5’s decoder and Sundial’s flow-matching head are not captured, so their depth claims concern part of the computation (Table 2).

Synthetic ground truth. The benchmark’s ground truth is exact only for the synthetic tier. Its generators encode our assumptions about what structure matters, and catch22 features may miss task-specific properties. The real-derived tier inherits real distributions but gives only attributional ground truth, and 27 of 76 concept parts in the example run are dominated by one real-derived generator. Whether a real corpus overlaps a model’s pretraining data cannot be verified.

What was confirmed. Held-out confirmation currently covers accuracy, geometry, perturbation agreement and input-level concept transfer. The causal concept results (families, sharing classes, shared-input agreement) replicate across two corpora but have not been tested on a sealed split, and should be read as exploratory. The 20 confirmed transfer claims of the v2 run come from two concepts. The v2 split’s exchangeability test was not persisted for this run (the v1 splits and two fresh epochs are exchangeable; MP-09).

SAE dependence. Concept results depend on the dictionary: its size, sparsity, revival recipe and seed. The system measures this dependence (seed-stability ceilings, fidelity, dead rates, reconstruction cost) but cannot remove it. With a fixed recipe, feature death is governed by layer geometry, not the recipe (alive atoms scale with effective dimension with

exponent 1.02; MP-05), so a single SAE configuration is not equally informative for every model.

Usability for novices. The report was designed for readers without an interpretability background, with plain-language explainers throughout. We have not run a user study, so claims about accessibility rest on design, not on measured comprehension.

The controls are not exhaustive. Section 8 lists failures that were caught. Controls cannot catch an error for which nobody has written a known-answer case, and the most dangerous failures in the table produced clean, confident output.

11 Conclusion

Interpretability of time-series foundation models is harder than it looks. Tokens mean different things in different models, there is no annotated ground truth, and the causal effects of internal features are small enough that measurement artifacts dominate unless they are controlled. TSFM-LENS addresses these problems with a controlled benchmark, measured time alignment, causal concepts gated by nulls and reach probes, and a register-then-confirm protocol. It packages all of this behind one command and a report written for non-specialists.

Applied to four architecturally distinct TSFMs, the system gives answers to our research questions that are both general and falsifiable. The models’ causal features act mainly on forecast level and shape, forming a continuum rather than discrete concepts. Different models detect the same things, and this replicates on sealed data, but they rarely use them the same way: shared effects on shared inputs are rare, convergent concepts dominate, and no concept is common to all four models. Individual features matter causally but account for little of the forecast. And the fourteen cases in which a plausible result was wrong show why these conclusions needed the controls that produced them. We hope the system, its example run and its ledger of findings, positive and negative, make such controlled, causal and confirmatory analysis the default for forecasting models.

Availability. Code (MIT license), configurations, the example run and the findings ledger are at <https://github.com/DanK10097110/TSFM-Interp>. The `main` branch holds the publishable system; the `dev` branch holds the full research log, study drivers and experiment configurations.

Acknowledgments and Disclosure of Funding

The author thanks the maintainers of the open checkpoints and libraries this work builds on (TimesFM, Chronos, Sundial, Timer, Lag-Llama, PyTorch, scikit-learn, zarr, plotly and catch22). **Funding and competing interests:** [to be completed by the author before submission].

Appendix A. Findings Ledger Index

The paper cites findings by identifier. Each identifier is an entry in `FINDINGS.md`, which records for every result its exact numbers, the models involved, its evidence class, its status (exploratory, confirmed, held-out confirmed, negative), a reproduction pointer (configuration, command and artifact path) and an interestingness score from 1 to 5. The ledger is curated from the research log on the `dev` branch and includes negative results on equal terms. Section letters: SH shared vs. unique representations; CA causal agreement; DE depth; PM per-model character; MP method positives; MN method negatives; BM benchmark trust. Table 7 indexes the entries cited here.

Table 7: Ledger entries cited in this paper (score = interestingness, 1–5).

ID	Claim (abridged)	Evidence / status	Score
SH-01	Peak CKA 0.38–0.47 above shuffled-series nulls (0.02–0.16) across five architectures, including Timer via the zero-code adapter	geometric; exploratory	5
SH-05	CKA and clustering AMI rank model pairs differently on three panels	geometric vs. descriptive; replicated	5
SH-10	Joint crosscoder loses to independent post-hoc-matched SAEs (0.2787 vs. 0.3204, $p = 0.002$), pre-registered	correlational; negative	5
SH-14	Concept transfer on shared inputs vs. stratum-matched null: 56.3% forward, 46.1% reciprocal; uniform null gave false 30/30	correlational; exploratory	5
SH-16	No atlas concept spans all four models; convergent is the modal sharing class	descriptive over causal parts; exploratory	5
SH-18	Registered concept-transfer claims confirmed on private data: 19/20, 20/20 frozen, 20/20 on v2	held-out confirmed	5
SH-19	Unit-level correspondence fails rotation controls (0.148 $\rightarrow$ 0.331); participation ratio as low as 1.54	geometric; negative	5
SH-20	11 of 13 targets non-modular once a minimum cluster size is enforced	descriptive; confirmed	4
CA-01	Perturbation-fingerprint agreement is pair-specific; TimesFM/Chronos-2 replicates on private data	causal within-model; held-out	4
CA-03	Seasonality circuit: 1 head (TimesFM) vs. 5 non-additive heads (Chronos-T5-Base)	causal within-model; exploratory	4
CA-04	Attention mass at seasonal lags uncorrelated with causal importance ($\rho = 0.026$, -0.118)	causal vs. descriptive; exploratory	4
CA-06	On shared inputs with matched floors, most apparent cross-model disagreement disappears (18/19 $\rightarrow$ 12/89)	causal on shared inputs; exploratory, 2 corpora	5
CA-07	Median level share of a causal feature’s effect 0.535; most features shape-causal beyond a level-removed null	causal within-model; exploratory	4
CA-10	Causal effect space is a continuum; families tile it but do not beat a shuffle null	descriptive; exploratory, 2 corpora	4
DE-01	Whole-layer patching hides a depth-intensifying recency effect in the last pre-horizon window	causal within-model; exploratory	4
DE-03	Chronos-2 skip lens was a silent no-op (identical MASE at 12 layers); corrected reach 0.737 $\rightarrow$ 0.000	causal within-model; confirmed	5
DE-04	Chronos-T5 encoder ends at depth 0.478 on a whole-stack axis; 50% comparable overlap with TimesFM	descriptive; confirmed	5
DE-05	Captured share of forecast FLOPs: 14.35% (Chronos-T5-Base) to 97.5% (Chronos-2)	descriptive; confirmed	5
DE-07	Chronos-T5 scaling ladder: only family decodability scales monotonically	mixed; exploratory	5

ID	Claim (abridged)	Evidence / status	Score
DE-08	Layer selection depends almost entirely on the selector (0/3 robust layers)	descriptive; confirmed	4
PM-05	Chronos-2 most accurate and best observed; winners vary by family; held-out on v2	behavioral; partly held-out	4
PM-07	Models differ $\sim 5.4\times$ in reversion to the mean; flat forecasts driven by weakly predictable data	behavioral; confirmed on run	4
PM-08	Random-weight twins are not one null: identity stack (TimesFM) vs. dead forecasts (Chronos-2/Bolt)	architecture fact; confirmed	5
PM-09	Sundial scorable as causal destination 1/71 $\rightarrow$ 12/71 after seeding fix	causal; corrected	3
PM-11	Causally load-bearing TimesFM random-walk concept fails seed stability	causal within-model; unstable	3
PM-13	Chronos-2 group attention unmeasured by design	scope statement	3
PM-15	Sundial adapter bypassed checkpoint normalization; residual 38.43 $\rightarrow$ 0.030	behavioral; fixed	4
MP-01	Token spans can be measured (IoU 1.0 on 6 checkpoints); zero-code adapter follows	method; confirmed	5
MP-02	Alignment gates must be normalized by their attainable ceiling	method; confirmed	3
MP-03	AuxK revival recovers dead SAE atoms 10–13 $\times$ on real activations	method; adopted	4
MP-04	Single-seed SAE gold ranking flips a bake-off verdict; 3 seeds fix it	method; fixed	5
MP-05	SAE feature death set by layer geometry (exponent 1.02), not recipe	correlational; replicated	5
MP-06	Ground-truth alignment of SAE features is real only over a permutation null	correlational; exploratory	3
MP-07	Ablation battery discriminates: 2.49–6.61 $\times$ chance	causal within-model; confirmed	4
MP-09	Dev and private splits exchangeable (energy $p = 0.51$; TOST 21/22)	held-out validation	3
MN-01	Similarity cannot detect lineage: untrained pair 0.878 > trained related pair 0.734	geometric; negative	5
MN-05	Residualizing on provenance made 93% of headline cards float-rounding noise	method; fixed	5
MN-06	A grounded small LLM narrator fabricates comparisons; guards do not converge	method; negative	5
MN-07	Causal fingerprints are nearly one-dimensional (PR 2.09)	causal, compared; negative	4
MN-14	64.4% of visually prominent SAE features are causally null	causal within-model	4
MN-15	SAE reconstruction moves the forecast $\sim 3.5\times$ more than removing a feature	causal within-model	4
MN-16	“Reconstruction beats clean forecast” replicated in 1/5 seeds; retracted	causal; retracted	4
MN-17	Generator-counterfactual concept mediation: 0/98 and 0/151	correlational; negative	5
MN-18	Absence of agreement is not disagreement: 109/288 $\rightarrow$ 9	method; fixed	4
MN-19	“Same inputs” must be top- k overlap, not whole-series correlation	method; fixed	4
MN-21	Reviving dead features hurts ground-truth alignment in TimesFM at every depth	correlational vs. causal; exploratory	5
MN-23	MASE denominator floors out on intermittent series	metric caveat	3
MN-24	A precision mismatch left aggregates intact but reorganized 71% of a partition	method; fixed	4
MN-25	Zero-code adapter resolves 2 of 8 probed TSFM checkpoints	descriptive; open	3
BM-05	Counterfactual identity check caught a corruption-replay bug (244/298 $\rightarrow$ 0/1113)	benchmark; fixed	2
BM-06	$\sim 42\%$ of the v1 dev corpus is weakly predictable real-derived data and drives the flat-forecast share	behavioral; measured	4

Appendix B. Statistical Details

Paired series bootstrap. For models a, b and series $s = 1, \dots, n$ with per-series scores $m_{a,s}, m_{b,s}$, let $d_s = m_{a,s} - m_{b,s}$ and $\bar{d} = \frac{1}{n} \sum_s d_s$. For $B = n_{\text{boot}}$ resamples $s^* \sim \text{Unif}\{1..n\}^n$, $\bar{d}_b^* = \frac{1}{n} \sum_i d_{s_i^*}$. The interval is the percentile interval of $\{\bar{d}_b^*\}$, and the two-sided p -value is $p = \min\left(1, \max\left(\frac{1}{B}, 2 \min\left\{\frac{1}{B} \#(\bar{d}_b^* \leq 0), \frac{1}{B} \#(\bar{d}_b^* \geq 0)\right\}\right)\right)$ (`analysis/stats.py::paired_bootstrap`). The floor at $1/B$ follows Phipson and Smyth (2010).

Cluster bootstrap for layer statistics. Statistics computed from window-level activations (CKA, fingerprints) resample series and keep all windows of a drawn series together, so dependence within a series is preserved (Cameron et al., 2008).

Holm and BH. For m sorted p -values $p_{(1)} \leq \dots \leq p_{(m)}$, Holm’s adjusted values are $\tilde{p}_{(i)} = \max_{j \leq i} \min(1, (m - j + 1)p_{(j)})$ (Holm, 1979). With every $p \geq 1/B$, the smallest attainable adjusted value is m/B ; the multiplicity ledger reports it for every family and flags families with $m/B > \alpha$. BH adjusted values are $\tilde{q}_{(i)} = \min_{j \geq i} \min(1, \frac{m}{j} p_{(j)})$ (Benjamini and Hochberg, 1995).

Row-matched ablation null. For feature i with row set R_i and J null directions $u_j \sim \mathcal{N}(0, I)$, $u_j \leftarrow u_j / \|u_j\|$, the null forecast removes $\bar{a} W_d u_j$ from the reconstruction, where $\bar{a}$ is the mean absolute activation removed by the ablations in the same batch. Channel c clears if $\frac{1}{|R_i|} \sum_{r \in R_i} |\Delta_{i,r}^{(c)}| > q_{0.95}\{|\Delta_{u_j,r}^{(c)}| : j \leq J, r \in R_i\}$. A null with no spread (all draws exactly 0, which quantized decoding can produce) is reported as degenerate and not scored.

Transfer AUC and its null. For a source concept with top- k series S and a destination layer with series-level feature activations a_f , the transfer statistic is $\max_f \text{AUC}(a_f; S)$, the best separation of S from the other series by any destination feature. Its null draws random sets of k series matched on the strata of S and recomputes the same maximum, so the search over features is included in the null (a per-feature null would ignore the search and clear almost everything). The reverse leg takes the chosen destination feature’s own top- k series and scores them by the source concept’s activation, against matched draws of that single statistic. Permutation p -values are $(1 + \#\{\text{null} \geq \text{obs}\}) / (1 + n)$. A registered claim combines the forward and reverse legs by $\max(p, p_{\text{rev}})$ (intersection–union) and is Holm-corrected across claims.

Shared-input agreement. On the shared set U , both sides are ablated and scored with the nine-channel battery, with level and shape separated. Statistic (i) is the Spearman concordance, across the series of U , of the two sides’ signed level effects; statistic (ii) is the cosine between the two sides’ level-removed shape vectors, each channel normalized by that side’s own null $q_{0.95}$. For the floors, each side draws random feature sets matched to its own activation on U , ablates them and re-scores the same statistic against the other side’s fixed, measured effect. A statistic agrees if it exceeds the $q_{0.95}$ of both sides’ floors; “acts differently” requires falling below the $q_{0.05}$ of both (`sae/shared_input_agreement.py`).

Minimum detectable effect. For a registered accuracy claim with private sample size n , the confirm stage resamples per-series paired differences, recentred and shifted by a candidate

effect, applies the same paired bootstrap test to each simulated sample, and bisects for the smallest shift detected with power 0.8 at α (capped at four standard deviations, reported as a boundary case), all *before* the private result is read (`analysis/power.py`).

Appendix C. Reproducibility

Artifacts. The example run `examples/concept_atlas_v2` contains the resolved configuration, the rendered report, and the lightweight artifacts of every stage (JSON and parquet; activations and SAE weights are omitted for size). `paper/figures/make_figures.py` regenerates Figures 11, 8, 10 and 9 from those artifacts, and accepts `--run` to point at any other run directory.

Commands. From `tsfm_lens/`:

```
python run.py --config configs/concept_atlas_v2.yaml          # full run
python run.py --config configs/concept_atlas_v2.yaml --stages report
python run.py --verify-provenance runs/concept_atlas_v2      # env diff
python -m pytest tests -q                                    # test suite
```

Building the v2 benchmark uses `tsfm_benchmark/configs/README.md`. The private split used here was opened once (2026-09-28); replicating the confirmation requires minting a fresh epoch.

Environment. Python 3.12, numpy 2.1.0, PyTorch 2.9.1 (CUDA 13.0), transformers 4.57.6, timesfm 2.0.2, chronos-forecasting 2.3.1, zarr 2.18.7 (zarr 3 stores read back empty under the v2 API) and scikit-learn 1.9.0; the full pin list and the reason for each load-bearing pin is `DEPENDENCIES.md`. Each run records its environment in `run_manifest.json`. The main-branch test suite passes (2,112 passed, 27 skipped for `tsfm_lens`; 124 passed, 1 skipped for `tsfm_benchmark`).

Runtime. Table 8 gives the wall-clock time per stage for the example run on one NVIDIA RTX A5000 (24 GB) with four CPU threads.

Stage	seconds	Stage	seconds	Stage	seconds
corpus	1.5	internals	230.5	attention	34.7
extract	57.5	lens	12.9	cluster	22.3
budget	5.3	l1	30.2	sae	8,593.9
frontend	8.6	l2	86.0	concepts	13,376.7
layer_screen	95.4	l3	195.0	exemplars, register	2.5
l0	207.5	confirm	131.0	report	59.5
Total					23,151 (6.43 h)

Table 8: Per-stage wall-clock time of the example run (`run_manifest.json`). The concept stage includes the shared-input agreement test (10,855 s) and three replicate SAE seeds per target (1,768 s).

Appendix D. Report Walkthrough

Figures 12–14 are unedited screenshots of the example report (`examples/concept_atlas_v2/report.html`), rendered with a headless browser.

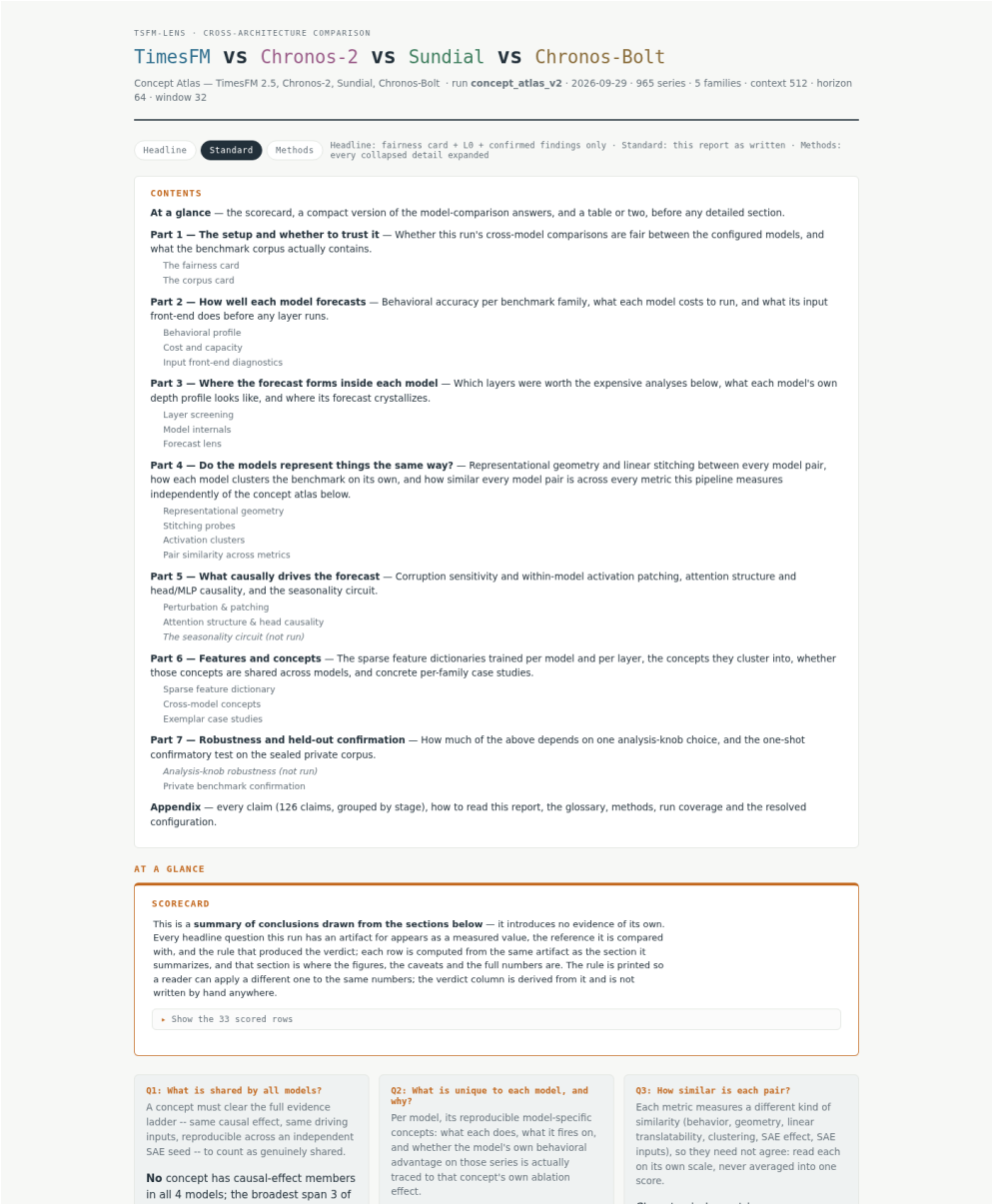
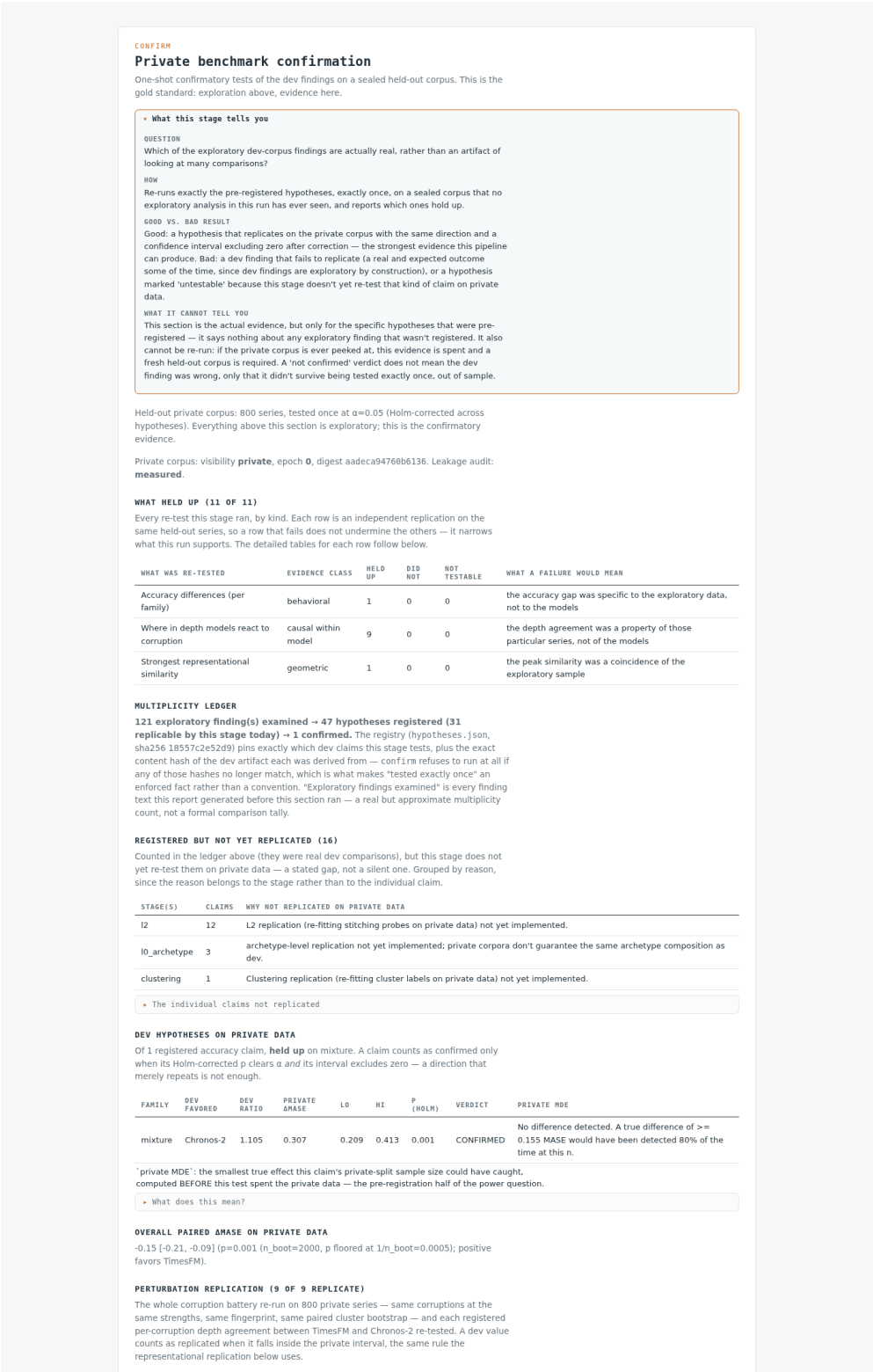


Figure 12: Report opening: reading-level toggle (headline, standard, methods), the question-first table of contents and the start of the derived scorecard and the three model-comparison answer boxes.



Figure 13: A stage section (L3, perturbation and patching). The open card states the question, the method, what a good and a bad result look like and what the analysis cannot tell you; each figure has a caption and a collapsible “What does this mean?” note.



Appendix E. Adapter Pitfalls: a Worked Case

The adapters guide (`tsfm_lens/docs/ADDING_A_MODEL.md`) documents every pitfall we met. The most instructive is the Sundial normalization bug (PM-15), because the pipeline measured it correctly and it was still misread.

What happened. Many Hugging Face checkpoints normalize inputs only inside their own `generate()` path. Sundial z-scores each series in `generate()`, but its `forward()` defaults to no normalization. `generate()` was broken on the installed transformers version, so the adapter called `forward()` directly and fed the model raw-scale input. The remote code’s own normalizing branch of `forward()` fails when more than one sample is drawn.

Symptoms. The `frontend` stage reported a scale-equivariance residual of 38.43 context standard deviations at scale 0.001 (the other models: at most 0.009), and forecasts on large-valued electricity series were 100% flat (MASE 4.134, against 24–32% flat for the other models). The residual was first recorded as a property of the model.

Fix and lesson. The adapter now normalizes in `prepare()` with the checkpoint’s exact rule (standard deviation plus 10^{-5}) and denormalizes samples in `predict()`, in one place, so capture, prediction and patching see identical inputs. The residual fell to 0.030 and the flat share on electricity to 0.08; the same defect in the zero-code adapter on Timer fell from 49.73 to 0.020. The report now flags any model whose residual exceeds $\max(1, 10 \times \text{the peers' median})$. The lesson generalizes: an adapter that calls a model below its public API must reproduce everything that API does to the input and output.

Checklist. The guide’s checklist, each item with a reason and a check: input normalization (frontend residual); seeded sampling for sampled heads, with identical seeds in every compared pair of calls (two unpatched predictions must match); clean-cache precision matching `predict()`; no silent batch chunking during patching; anchored layer regular expressions; contiguous token spans (measured by span discovery, verified by the alignment gate); routing of non-time-localized models; stating which part of an encoder–decoder is captured; reach controls (self-patch exactly 0, cross-patch nonzero); random-weight twins checked for reach before use as a causal floor; original-scale outputs; rerunning `--discover-layers`, `--check-alignment` and `--check-adapter` after any library or checkpoint change; and adapter tests with a planted answer and a decoy.

References

- J. Adebayo, J. Gilmer, M. Muelly, I. Goodfellow, M. Hardt, and B. Kim. Sanity checks for saliency maps. In *Advances in Neural Information Processing Systems*, 2018.
- G. Alain and Y. Bengio. Understanding intermediate layers using linear classifier probes. In *International Conference on Learning Representations (Workshop Track)*, 2017.
- A. F. Ansari, L. Stella, C. Turkmen, X. Zhang, P. Mercado, H. Shen, O. Shchur, S. S. Rangapuram, S. Pineda Arango, S. Kapoor, J. Zschiegner, D. C. Maddix, H. Wang, M. W. Mahoney, K. Torkkola, A. G. Wilson, M. Bohlke-Schneider, and Y. Wang. Chronos: Learning the language of time series. *Transactions on Machine Learning Research*, 2024.

- A. F. Ansari, O. Shchur, et al. Chronos-2: From univariate to universal forecasting. *arXiv preprint arXiv:2510.15821*, 2025.
- Y. Bansal, P. Nakkiran, and B. Barak. Revisiting model stitching to compare neural representations. In *Advances in Neural Information Processing Systems*, 2021.
- Y. Belinkov. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1):207–219, 2022.
- N. Belrose, Z. Furman, L. Smith, D. Halawi, I. Ostrovsky, L. McKinney, S. Biderman, and J. Steinhardt. Eliciting latent predictions from transformers with the tuned lens. *arXiv preprint arXiv:2303.08112*, 2023.
- Y. Benjamini and Y. Hochberg. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B*, 57(1): 289–300, 1995.
- J. Bloom, C. Tigges, A. Duong, and D. Chanin. SAELens. <https://github.com/jbloomAus/SAELens>, 2024.
- T. Bolukbasi, A. Pearce, A. Yuan, A. Coenen, E. Reif, F. Viégas, and M. Wattenberg. An interpretability illusion for BERT. *arXiv preprint arXiv:2104.07143*, 2021.
- T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, T. Conerly, N. Turner, C. Anil, C. Denison, A. Askell, et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023.
- A. C. Cameron, J. B. Gelbach, and D. L. Miller. Bootstrap-based improvements for inference with clustered errors. *The Review of Economics and Statistics*, 90(3):414–427, 2008.
- H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models. In *International Conference on Learning Representations*, 2024.
- A. Das, W. Kong, R. Sen, and Y. Zhou. A decoder-only foundation model for time-series forecasting. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 2024.
- F. Divo, M. Kraus, R. Härle, P. Ahrend, and K. Kersting. Finding the Zeitgeist in time series foundation models. In *ICLR 2026 Workshop on Time Series in the Age of Large Models*, 2026.
- S. Dooley, G. S. Khurana, C. Mohapatra, S. Naidu, and C. White. ForecastPFN: Synthetically-trained zero-shot forecasting. In *Advances in Neural Information Processing Systems*, 2023.
- C. Dwork, V. Feldman, M. Hardt, T. Pitassi, O. Reingold, and A. Roth. The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349(6248):636–638, 2015.
- B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall/CRC, 1993.

- V. Ekambaram, A. Jati, P. Dayama, S. Mukherjee, N. H. Nguyen, W. M. Gifford, C. Reddy, and J. Kalagnanam. Tiny time mixers (TTMs): Fast pre-trained models for enhanced zero/few-shot forecasting of multivariate time series. In *Advances in Neural Information Processing Systems*, 2024.
- N. Elhage, T. Hume, C. Olsson, N. Schiefer, T. Henighan, S. Kravec, Z. Hatfield-Dodds, R. Lasenby, D. Drain, C. Chen, R. Grosse, S. McCandlish, J. Kaplan, D. Amodei, M. Wattenberg, and C. Olah. Toy models of superposition. *Transformer Circuits Thread*, 2022.
- J. Fiotto-Kaufman, A. R. Loftus, E. Todd, J. Brinkmann, K. Pal, D. Troitskii, M. Ripa, A. Belfki, C. Rager, C. Juang, et al. NNsight and NDIF: Democratizing access to open-weight foundation model internals. In *International Conference on Learning Representations*, 2025.
- L. Gao, T. Dupré la Tour, H. Tillman, G. Goh, R. Troll, A. Radford, I. Sutskever, J. Leike, and J. Wu. Scaling and evaluating sparse autoencoders. In *International Conference on Learning Representations*, 2025.
- P. Gao, E. Trautmann, B. Yu, G. Santhanam, S. Ryu, K. Shenoy, and S. Ganguli. A theory of multineuronal dimensionality, dynamics and measurement. *bioRxiv*, page 214262, 2017.
- A. Geiger, H. Lu, T. Icard, and C. Potts. Causal abstractions of neural networks. In *Advances in Neural Information Processing Systems*, 2021.
- A. Gelman and E. Loken. The statistical crisis in science. *American Scientist*, 102(6): 460–465, 2014.
- T. Gneiting and A. E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007.
- R. Godahewa, C. Bergmeir, G. I. Webb, R. J. Hyndman, and P. Montero-Manso. Monash time series forecasting archive. In *Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- M. Goswami, K. Szafer, A. Choudhry, Y. Cai, S. Li, and A. Dubrawski. MOMENT: A family of open time-series foundation models. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- S. Heimersheim and N. Nanda. How to use and interpret activation patching. *arXiv preprint arXiv:2404.15255*, 2024.
- J. Hewitt and P. Liang. Designing and interpreting probes with control tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019.
- S. Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- M. Huh, B. Cheung, T. Wang, and P. Isola. Position: The platonic representation hypothesis. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

- R. J. Hyndman and A. B. Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006.
- A. A. Ismail, M. Gunady, H. Corrada Bravo, and S. Feizi. Benchmarking deep learning interpretability in time series predictions. In *Advances in Neural Information Processing Systems*, 2020.
- S. Jain and B. C. Wallace. Attention is not explanation. In *Proceedings of NAACL-HLT*, 2019.
- M. Kalnāre, S. Kitharidis, T. Bäck, and N. van Stein. Mechanistic interpretability for transformer-based time series classification. *arXiv preprint arXiv:2511.21514*, 2025.
- Y. Kang, R. J. Hyndman, and F. Li. GRATIS: Generating time series with diverse and controllable characteristics. *Statistical Analysis and Data Mining*, 13(4):354–376, 2020.
- S. Kapoor and A. Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9), 2023.
- T. Kim, J. Kim, Y. Tae, C. Park, J.-H. Choi, and J. Choo. Reversible instance normalization for accurate time-series forecasting against distribution shift. In *International Conference on Learning Representations*, 2022.
- N. Kokhlikyan, V. Miglani, M. Martin, E. Wang, B. Alsallakh, J. Reynolds, A. Melnikov, N. Kliushkina, C. Araya, S. Yan, and O. Reblitz-Richardson. Captum: A unified and generic model interpretability library for PyTorch. *arXiv preprint arXiv:2009.07896*, 2020.
- S. Kornblith, M. Norouzi, H. Lee, and G. Hinton. Similarity of neural network representations revisited. In *Proceedings of the 36th International Conference on Machine Learning*, 2019.
- N. Kriegeskorte, M. Mur, and P. Bandettini. Representational similarity analysis – connecting the branches of systems neuroscience. *Frontiers in Systems Neuroscience*, 2:4, 2008.
- H. R. Künsch. The jackknife and the bootstrap for general stationary observations. *The Annals of Statistics*, 17(3):1217–1241, 1989.
- K. Lenc and A. Vedaldi. Understanding image representations by measuring their equivariance and equivalence. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015.
- Y. Li, J. Yosinski, J. Clune, H. Lipson, and J. Hopcroft. Convergent learning: Do different neural networks learn the same representations? In *International Conference on Learning Representations*, 2016.
- J. Lindsey, A. Templeton, J. Marcus, T. Conerly, J. Batson, and C. Olah. Sparse crosscoders for cross-layer features and model diffing. *Transformer Circuits Thread*, 2024.
- Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023.

- Y. Liu, H. Zhang, C. Li, X. Huang, J. Wang, and M. Long. Timer: Generative pre-trained transformers are large time series models. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- Y. Liu, G. Qin, Z. Shi, Z. Chen, C. Yang, X. Huang, J. Wang, and M. Long. Sundial: A family of highly capable time series foundation models. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025. arXiv:2502.00816.
- C. H. Lubba, S. S. Sethi, P. Knaute, S. R. Schultz, B. D. Fulcher, and N. S. Jones. catch22: CAnonical time-series characteristics. *Data Mining and Knowledge Discovery*, 33(6): 1821–1852, 2019.
- A. Makelov, G. Lange, A. Geiger, and N. Nanda. Is this the subspace you are looking for? an interpretability illusion for subspace activation patching. In *International Conference on Learning Representations*, 2024.
- A. Makhzani and B. Frey. k-sparse autoencoders. *arXiv preprint arXiv:1312.5663*, 2013.
- K. Meng, D. Bau, A. Andonian, and Y. Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, 2022.
- A. Mishra. Dissecting Chronos: Sparse autoencoders reveal causal feature hierarchies in time series foundation models. In *ICLR 2026 Workshop on Time Series in the Age of Large Models*, 2026. arXiv:2603.10071.
- A. S. Morcos, M. Raghu, and S. Bengio. Insights on representational similarity in neural networks with canonical correlation. In *Advances in Neural Information Processing Systems*, 2018.
- N. Nanda and J. Bloom. TransformerLens. <https://github.com/TransformerLensOrg/TransformerLens>, 2022.
- Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *International Conference on Learning Representations*, 2023.
- B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018.
- nostalgebraist. interpreting GPT: the logit lens. LessWrong, 2020.
- C. Olah, N. Cammarata, L. Schubert, G. Goh, M. Petrov, and S. Carter. Zoom in: An introduction to circuits. *Distill*, 2020. doi: 10.23915/distill.00024.001.
- K. Oublal, Q. Bouniot, Q. Gan, S. Cl  men  on, and Z. Akata. TimeSAE: Causal sparse decoding for faithful explanations of black-box time series models. *arXiv preprint arXiv:2601.09776*, 2026.
- G. Paulo and N. Belrose. Sparse autoencoders trained on the same data learn different features. *arXiv preprint arXiv:2501.16615*, 2025.

- B. Phipson and G. K. Smyth. Permutation p-values should never be zero: Calculating exact p-values when permutations are randomly drawn. *Statistical Applications in Genetics and Molecular Biology*, 9(1), 2010.
- C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.
- M. Raghu, J. Gilmer, J. Yosinski, and J. Sohl-Dickstein. SVCCA: Singular vector canonical correlation analysis for deep learning dynamics and interpretability. In *Advances in Neural Information Processing Systems*, 2017.
- K. Rasul, A. Ashok, A. R. Williams, H. Ghonia, R. Bhagwatkar, A. Khorasani, M. J. Darvishi Bayazi, G. Adamopoulos, R. Riachi, N. Hassen, M. Biloš, S. Garg, A. Schneider, N. Chapados, A. Drouin, V. Zantedeschi, Y. Nevmyvaka, and I. Rish. Lag-Llama: Towards foundation models for probabilistic time series forecasting. *arXiv preprint arXiv:2310.08278*, 2023.
- B. Recht, R. Roelofs, L. Schmidt, and V. Shankar. Do ImageNet classifiers generalize to ImageNet? In *Proceedings of the 36th International Conference on Machine Learning*, 2019.
- P. J. Rousseeuw. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987.
- H. Sakoe and S. Chiba. Dynamic programming algorithm optimization for spoken word recognition. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 26(1):43–49, 1978.
- D. J. Schuirmann. A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of Pharmacokinetics and Biopharmaceutics*, 15(6):657–680, 1987.
- X. Shi, S. Wang, Y. Nie, D. Li, Z. Ye, Q. Wen, and M. Jin. Time-MoE: Billion-scale time series foundation models with mixture of experts. In *International Conference on Learning Representations*, 2025.
- U. Simonsohn, J. P. Simmons, and L. D. Nelson. Specification curve analysis. *Nature Human Behaviour*, 4:1208–1214, 2020.
- G. J. Székely and M. L. Rizzo. Energy statistics: A class of statistics based on distances. *Journal of Statistical Planning and Inference*, 143(8):1249–1272, 2013.
- A. Templeton, T. Conerly, J. Marcus, J. Lindsey, T. Bricken, B. Chen, A. Pearce, C. Citro, E. Ameisen, A. Jones, et al. Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet. *Transformer Circuits Thread*, 2024.
- J. Vig, S. Gehrmann, Y. Belinkov, S. Qian, D. Nevo, Y. Singer, and S. Shieber. Investigating gender bias in language models using causal mediation analysis. In *Advances in Neural Information Processing Systems*, 2020.

- N. X. Vinh, J. Epps, and J. Bailey. Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance. *Journal of Machine Learning Research*, 11:2837–2854, 2010.
- S. Wan, M. Ma, H. Zhu, G. Liu, S. Wang, Q. Wen, and H. Liu. Forecast collapse in time-series foundation models. *arXiv preprint arXiv:2608.14106*, 2026.
- K. Wang, A. Variengien, A. Conmy, B. Shlegeris, and J. Steinhardt. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. In *International Conference on Learning Representations*, 2023.
- M. Wiliński, M. Goswami, W. Potosnak, N. Żukowska, and A. Dubrawski. Exploring representations and interventions in time series foundation models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, 2025.
- G. Woo, C. Liu, A. Kumar, C. Xiong, S. Savarese, and D. Sahoo. Unified training of universal time series forecasting transformers. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- Z. Wu, A. Geiger, A. Arora, J. Huang, Z. Wang, N. D. Goodman, C. D. Manning, and C. Potts. pyvene: A library for understanding and improving PyTorch models via interventions. In *Proceedings of NAACL-HLT (System Demonstrations)*, 2024.
- A. Yıldırım. Superposition is not necessary: A mechanistic interpretability analysis of transformer representations for time series forecasting. *arXiv preprint arXiv:2605.05151*, 2026.
- F. Zhang and N. Nanda. Towards best practices of activation patching in language models: Metrics and methods. In *International Conference on Learning Representations*, 2024.
- B. Zhou, Y. Sun, D. Bau, and A. Torralba. Revisiting the importance of individual units in CNNs via ablation. *arXiv preprint arXiv:1806.02891*, 2018.
- H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, 2021.